The Engine's Mind

A Self-Tutor in Neural Monte-Carlo Tree Search

From one slot machine to Leela's suppressed sacrifices

Written for the Chess Speak Out Loud project

Draft of August 6, 2026

How to use this book

What this is

This is a tutor, not a summary. It teaches one thing: **how a neural chess engine thinks, from the ground up, in enough detail that you could rebuild it.** By the end you should be able to look at a line of Leela's verbose output — something like

```
info string b3b8 (N: 214) (P: 3.11%) (Q: 0.99871) (U: 0.04102)
```

and read it the way you read a chess diagram: instantly, structurally, knowing what every number is, where it came from, what would change it, and what it would look like if it were lying to you.

We get there by building. Nothing is asserted and left standing. Every formula in this book is constructed in front of you — usually starting from a version that is *wrong*, watching it fail on a concrete example, and repairing exactly the part that failed. The famous PUCT formula that governs Leela's every decision,

$$\arg\max_a \Big[\underbrace{Q(s,a)}_{\text{what I've measured}} + \underbrace{c_{\text{puct}}\, P(a \mid s)\, \frac{\sqrt{N(s)}}{1+N(s,a)}}_{\text{what I haven't checked yet}} \Big]$$

is not introduced. It is *assembled*, in Chapter 6, out of five failed attempts, each of which teaches you what one symbol is for. If you skip to the end and read the formula, you have learned nothing. That is exactly the failure mode this book exists to fix.

Who it is for

You. A strong club player (2100–2200) who directs an engine-based training project, who wants the theory properly rather than as a listicle, and who has been burned by plausible-sounding explanations that turned out to be hollow.

Assumed: chess to a strong club standard; secondary-school algebra (you can rearrange $y = mx + c$ and you are not frightened by a summation sign).

Not assumed: calculus, probability, linear algebra, machine learning, or programming. Every mathematical tool used is built here, in the order it is needed. Where calculus is genuinely required (once, for gradient descent, in Chapter 10) it is developed from the idea of a slope on a hill, with numbers.

The seven rules this book follows

1. **Every formula is built, never dropped.** A formula arrives as version 0 (crude, obviously wrong), is broken on an example, and is repaired. You see the scar tissue. That

is the point: the final formula's strange parts — why $\sqrt{N}$ and not N, why $1 + N(s, a)$ and not $N(s, a)$ — are exactly the repairs.

2. **Every idea gets arithmetic.** Not "the bonus decays"; instead, a table where the bonus is 0.55, then 0.28, then 0.19, and you can check the division yourself. Understanding that survives contact with a real engine log is arithmetic-shaped.

3. **The engine numbers in this book are real.** Every table labelled *Real engine output* was produced by running the actual `lc0.exe` and network in `engine/` on this machine, by the script `tools/collect_engine_data.py`. Nothing is typed from memory or imagination. Where a number is invented for the sake of clean arithmetic, it says so in the same breath. See "The honesty apparatus" below — this matters more than it sounds.

4. **Chess claims are verified by an engine, not asserted.** This book teaches computer science, but it uses chess positions. Any chess claim ("this wins", "this throws away the win") has been checked with Stockfish at depth 30 and the check is reported. The author of the text is not a chess authority and does not pretend to be.

5. **Exercises are part of the exposition, not decoration.** Some results are *only* developed in the exercises. Full solutions are in Appendix B, worked at the same level of detail as the text. Do them with a pen. The arithmetic is deliberately small enough to do by hand.

6. **Confidence is labelled.** Textbook mathematics, published research findings, and this project's own untested hypotheses are three different things, and this book never lets them wear the same clothes.

7. **Everything lands somewhere in your repository.** Each chapter ends by pointing at the file in `chess_speak_out_loud` where the idea actually lives, or notes honestly that it does not exist yet. Appendix E is the full map.

The honesty apparatus

This project has been burned twice by fluent, plausible, fabricated content — a "sacrifice" metric that never checked material, and a batch of invented master annotations. The doctrine that came out of it (*a bad coach does more harm than no coach*) applies to a textbook at least as much as to a chess tutor. So claims in this book are colour-coded by how much weight they will bear:

Confidence: established

Mathematics or documented engine behaviour. True by proof or by reading the source. If this is wrong, the error is mine and it is a plain error.

Confidence: published research finding

A published experimental result from the interpretability literature. Real evidence, real peer scrutiny — but obtained on a *particular* network, with a particular method, and not automatically true of the exact net in `engine/`. Reproduce before relying on it.

Confidence: this project's hypothesis — not yet verified

An idea this project intends to build, which nobody has yet demonstrated works here. It may be excellent. It is not knowledge. Where a chapter proposes one, it also proposes the experiment that could kill it.

Real engine output

Numbers produced by running the engine in `engine/` while writing this book. The exact command line is in Appendix D; re-run it and you should get the same table (the search is deterministic at fixed node counts with one thread).

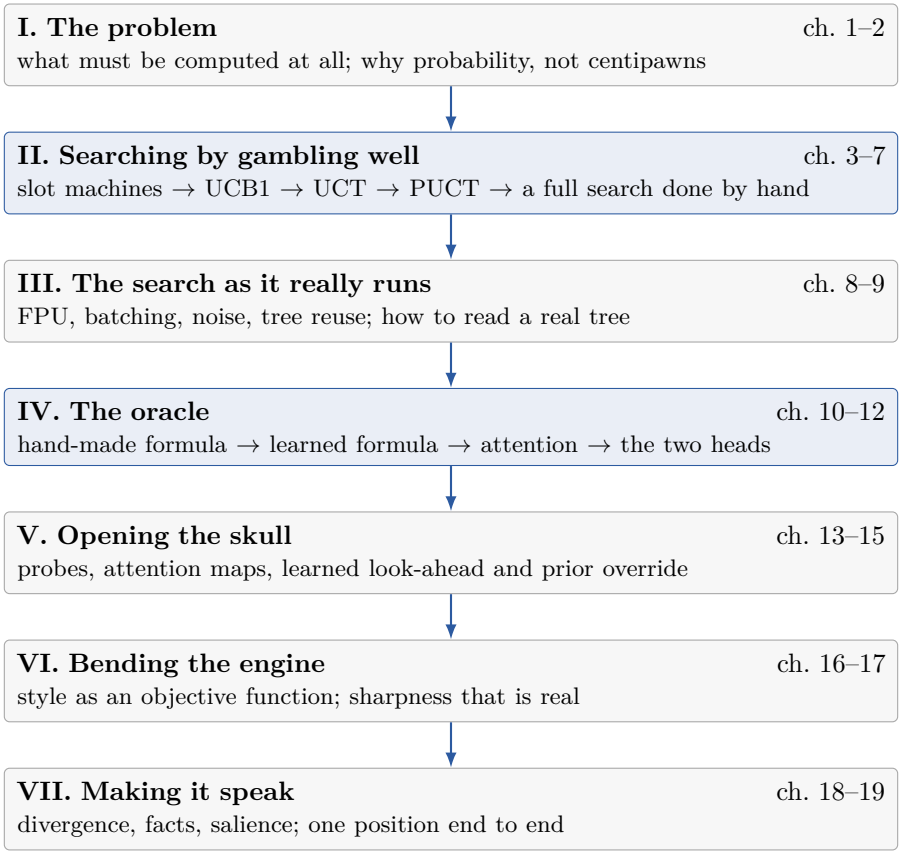
And, for the fourth case:

Worked example 0.1 — invented numbers, and why that is fine here

When the point is the *arithmetic of the algorithm*, real numbers are often clumsy — $P = 0.4413$ obscures the division you are meant to see. In those places this book uses round invented numbers and says so in the caption: *illustrative, not engine output.* The rule is simple: invented numbers are allowed to demonstrate a *mechanism*; they are never allowed to support a *claim about chess or about Leela.*

How to read it

The book has seven parts and a spine. The spine is Parts II and IV: the search (how the engine decides where to spend its thinking) and the network (how it forms an opinion in the first place). Everything else hangs off those.



Three ways through.

- **Cover to cover** (the intended path). Parts build strictly on each other. Chapter 7 — a complete search executed by hand with real priors — is the point where the first half either clicks or does not; do not pass it until it does.

- **Search first, network later.** Chapters 1–9 are self-contained if you accept the network as a black box that returns two things. Many people find the search easier and the network more mysterious; this order lets the mystery wait.
- **Reference.** Appendix A is a one-page symbol table, Appendix C a glossary, Appendix E the map from concept to source file. Chapter 9 is the one to reread whenever you are staring at a confusing engine log.

Pace. There are 19 chapters and roughly 130 exercises. At one chapter per sitting this is about three weeks. The two long chapters (7 and 11) are each worth two sittings on their own.

A note on what this book will not do

It will not do chess analysis. When a position appears, its role is to make an algorithm concrete, and any evaluative statement about it is either (a) an engine output, quoted and attributed, or (b) elementary and flagged as such. The governing rule of the project it was written for — *the engine is the chess authority; language is a translator, never a reasoner* — is also the rule this book obeys about itself.

Contents

Part I

The Problem, and the Two Questions

Chapter 1

What a chess engine is actually trying to do

After this chapter you can

- State the engine problem as exactly two questions, and show that every engine ever built is a different answer to those two questions.
- Compute, from your own 9,030-game corpus, why brute force is not merely slow but physically impossible.
- Run minimax and alpha–beta by hand on a small tree, and count the nodes each visits.
- Write down the classical evaluation function $f(s) = \sum_i w_i \phi_i(s)$ and name precisely the two things it cannot do.
- Explain why Leela replaces *both* halves of the classical answer, not just one.

1.1 Two questions, and only two

Put a position in front of any chess-playing machine ever built — Turing's paper machine of 1948, Deep Blue, Stockfish, Leela — and it must answer two questions.

Key idea

Question 1 (evaluation). *How good is this position?* Reduce a board to a number, or to something a number can be extracted from.

Question 2 (search). *Where should I spend my thinking?* There are more continuations than can ever be examined. Which ones get examined?

That is the whole subject. Everything in the rest of this book — transformers, attention heads, Monte-Carlo trees, linear probes — is machinery bolted onto one of those two questions. It is worth fixing the shape of the problem in your head now, because the distinctive thing about Leela is easy to state once you have it:

	Answer to Q1 (evaluate)	**Answer to Q2 (where to look)**
Deep Blue, Stockfish (classical)	hand-written formula	alpha–beta, hand-written ordering
Stockfish (modern, NNUE)	small neural network	alpha–beta, hand-written ordering
Leela / AlphaZero	**neural network**	**neural network** + Monte-Carlo tree

Leela's real novelty is the right-hand column. Plenty of engines learned their evaluation. Leela is the family that also *learned where to look* — and that single fact is what makes it interpretable in a way Stockfish is not, and therefore what makes your whole project possible. When you ask Leela "what were you considering?", it has an answer, because considering is something it does with a learned opinion rather than with a fixed procedure.

1.2 Why you cannot simply look at everything

The naive answer to Question 2 is: look at everything. Let us kill it with real numbers — yours.

1.2.1 The branching factor, measured on your own games

The **branching factor** b is how many legal moves there are, on average, in a position. Textbooks say "about 30" and rarely say where the number came from. Here it is, measured on the corpus in `games_of_derdiedasdie/`:

Real engine outputtitle

Games	9,030
Positions examined	549,816
Mean legal moves per position	**31.35**
Median legal moves	33
Most legal moves seen in any position	111
Fewest (forced move)	1
Mean game length	60.9 plies
Mean legal moves, opening (moves 1–12)	32.84
Mean legal moves, middlegame	32.48
Mean legal moves, endgame ($\leq$ 12 pieces)	15.81

So $b \approx 31$ in your chess. Note the shape of that table: the branching factor is essentially flat at ≈ 32 through opening and middlegame and then halves in the endgame. That is one reason engines feel "deeper" in endgames — the tree is genuinely narrower there, so the same node budget buys more plies.

Notation

A **ply** is one move by one side. "White plays $e4$" is one ply. A *move* in chess notation ($1.e4\ e5$) is two plies. Engine depth is always counted in plies. In this book, d is depth in plies.

1.2.2 The count

If every position has b replies, then looking d plies ahead exhaustively means visiting about

$$T(d) \;=\; b^d \tag{1.1}$$

leaf positions. Put $b = 31$ in and turn the crank:

Depth d (plies)	In chess terms	Positions 31^d
2	one move each	961
4	two moves each	$\approx 9.2 \times 10^5$
6	three moves each	$\approx 8.9 \times 10^8$
8	four moves each	$\approx 8.5 \times 10^{11}$
10	five moves each	$\approx 8.2 \times 10^{14}$
20	ten moves each	$\approx 6.7 \times 10^{29}$
40	a short game	$\approx 4.5 \times 10^{59}$

Worked example 1.1 — how long would depth 20 take?

Suppose you evaluate a billion (10^9) positions per second — generous; a strong classical engine on good hardware manages a few tens of millions.

$$\frac{6.7 \times 10^{29} \text{ positions}}{10^9 \text{ positions/second}} = 6.7 \times 10^{20} \text{ seconds.}$$

A year is about 3.15×10^7 seconds, so this is

$$\frac{6.7 \times 10^{20}}{3.15 \times 10^7} \approx 2.1 \times 10^{13} \text{ years,}$$

roughly *fifteen hundred times the age of the universe* — to see ten moves ahead once. And your games last thirty moves, not ten.

Shannon made this argument in 1950 and put the number of plausible chess games at about 10^{120}, now called the Shannon number. The exact exponent does not matter. What matters is the shape of Equation (1.1): the cost is *exponential* in depth, so hardware improvements buy almost nothing. A machine a thousand times faster buys you

$$\log_{31}(1000) \approx 2.0 \quad \text{extra plies.}$$

One extra move each. That is what a factor of a thousand is worth. Every serious idea in engine design is therefore an idea about *not looking at things*.

Key idea

Exponential growth means the interesting question is never "how fast can I evaluate?" but "**how small can I make the effective branching factor?**" Write b_{eff} for the average number of moves an engine actually explores per node. Classical alpha–beta with good move ordering gets b_{eff} down to roughly $\sqrt{b} \approx 6$. Leela's answer — ask a network which moves are worth the visit — is a different and, in the positions this project cares about, more selective attack on the same quantity.

1.3 Minimax: the correct answer, if you could afford it

Before improving on brute force, be precise about what brute force computes.

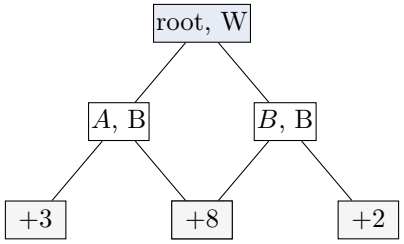
Chess is a two-player game with strictly opposed interests. If I score a position from White's point of view, then White wants the score high and Black wants it low. Suppose we can evaluate leaf positions somehow (that is Question 1; assume it for now). Then the value of an internal node is defined recursively:

$$V(s) \;=\; \begin{cases} f(s), & \text{if } s \text{ is a leaf (we stop here),} \\ \max_a V(s{\cdot}a), & \text{if White is to move at } s, \\ \min_a V(s{\cdot}a), & \text{if Black is to move at } s, \end{cases} \tag{1.2}$$

where $s{\cdot}a$ means "the position after playing move a in position s". This is **minimax**: it assumes both sides play the best move available inside the part of the tree we looked at.

Worked example 1.2 — minimax by hand

White to move at the root, two plies deep, two moves each. Leaf evaluations (in pawns, from White's point of view) are *invented for the arithmetic*:



Work from the bottom up.

- Node A: *Black* to move, so Black takes the minimum of $\{+3, -1\}$: $V(A) = -1$.
- Node B: Black takes the minimum of $\{+8, +2\}$: $V(B) = +2$.
- Root: *White* to move, takes the maximum of $\{-1, +2\}$: $V(\text{root}) = +2$, achieved by playing B.

Notice what just happened at node B: the $+8$ leaf — the most attractive number on the board — had *no influence whatsoever*. Black simply would not go there. A very common human error, and the one minimax is built to prevent, is letting a line's best-case leaf colour your judgement of the line. The value of a move is what the opponent will *allow*, not what you hope for.

1.3.1 The negamax trick (you will need it in Chapter 5)

Writing separate `max` and `min` cases is clumsy, and it becomes an actual bug factory later when values flow up a Monte-Carlo tree. The standard fix is to score every position **from the point of view of the side to move** instead of always from White's. Then both players are maximisers, and the only thing that happens when you cross a ply boundary is a change of sign:

$$V(s) \;=\; \max_a \big[-V(s{\cdot}a)\big]. \tag{1.3}$$

Read it aloud: *my value is the best I can do, where "good for me" is "bad for the person who moves next"*. Check it against the worked example: if $V(A) = -1$ from White's view and it is Black to move at A, then from Black's own point of view A is worth $+1$; White's root value through A is $-(+1) = -1$. Same answer, one rule.

Where people go wrong

This sign flip is responsible for a large fraction of all bugs in tree-search code, and for a large fraction of all confusion when reading engine output. Leela's Q values are side-to-move-relative: a Q of $+0.9$ in a node where Black is to move means *Black* is winning. Chapter 7

makes you flip signs by hand thirty-odd times, on purpose, until it stops being a thing you have to think about.

1.4 Alpha–beta: proving you do not need to look

Minimax visits every leaf. Alpha–beta pruning gets the identical answer while skipping large parts of the tree, using one observation: *as soon as a line is proven bad enough that the opponent will never allow it, stop analysing it.* You do this at the board already.

Worked example 1.3 — a cutoff, in chess words and in numbers

White to move; two candidate moves, A and B.

1. You analyse A fully and conclude it gives $+2$. So you now have a guarantee: the root is worth *at least* $+2$. Call that guarantee $\alpha = +2$.
2. You start on B and look at Black's first reply B_1, which gives $+1$.
3. Stop. Black chooses at B, so $V(B) \leq +1$. Black already has a way to hold B to $+1$, which is worse for you than the $+2$ you have banked. Whatever Black's *other* replies to B do, they can only lower $V(B)$ further. So B can never beat A, and the remaining replies to B — possibly millions of positions — need never be examined.

In chess: *"I don't need to check the rest of this line; he has a good answer already."*

Confidence: established

Alpha–beta returns exactly the same value as full minimax — it prunes only branches that provably cannot affect the root. With perfect move ordering it examines about $b^{d/2}$ leaves instead of b^d, i.e. the effective branching factor drops to $\sqrt{b}$ (Knuth & Moore, 1975). With $b = 31$, that is $b_{\text{eff}} \approx 5.6$: the difference between depth 8 and depth 16 on the same hardware.

Two consequences worth carrying forward:

1. **Alpha–beta's power comes from move ordering.** The $\sqrt{b}$ result assumes you try the best move first. Classical engines therefore spend enormous effort on *move-ordering heuristics* (try captures first, try the move that worked in a sibling node, etc.). Those heuristics are hand-written guesses about which move is good — which is to say: *classical engines already had a policy, they just wrote it by hand.* Leela's policy head is the learned version of exactly this. It is not a new kind of component; it is an old component that stopped being hand-written.
2. **Alpha–beta is all-or-nothing.** A branch is either searched or cut. There is no notion of "spend 3% of my effort on this suspicious-looking move". Monte-Carlo search in Part II replaces that binary with a *budget allocation*, and that difference is what makes Leela's search readable as a statement of interest — the visit counts tell you how seriously each plan was taken.

1.5 The classical answer to Question 1: a hand-written formula

Now Question 1. How did engines score a leaf before neural networks? By adding up features:

$$f(s) \;=\; \sum_{i=1}^{k} w_i \, \phi_i(s), \tag{1.4}$$

where each ϕ_i is a **feature** — a countable property of the position — and w_i is a hand-chosen **weight** saying how much that property is worth.

Worked example 1.4 — a complete (bad) evaluation function

The simplest usable evaluation is material only, with the classical values.

$$\begin{aligned}
\phi_1 &= (\text{White pawns}) - (\text{Black pawns}), & w_1 &= 1.0 \\
\phi_2 &= (\text{White knights}) - (\text{Black knights}), & w_2 &= 3.0 \\
\phi_3 &= (\text{White bishops}) - (\text{Black bishops}), & w_3 &= 3.2 \\
\phi_4 &= (\text{White rooks}) - (\text{Black rooks}), & w_4 &= 5.0 \\
\phi_5 &= (\text{White queens}) - (\text{Black queens}), & w_5 &= 9.0
\end{aligned}$$

In a position where White is a knight up for two pawns: $\phi_1 = -2$, $\phi_2 = +1$, rest zero, so

$$f(s) = 1.0(-2) + 3.0(+1) = +1.0.$$

"White is a pawn better." A real classical evaluation adds dozens more terms — passed pawns, king safety, rook on an open file, bishop pair, mobility — each with a weight someone tuned. Stockfish's hand-written evaluation, before it went neural, had hundreds.

Equation (1.4) is worth staring at, because **the neural network in Part iv is the same equation**. Really: the value head is $f(s) = \sum_i w_i \phi_i(s)$ where the ϕ_i are computed by the layers below rather than written by a person, and the w_i are found by gradient descent rather than tuned by hand. Chapter 10 makes that transition one step at a time. Nothing alien arrives.

1.5.1 The two things a hand-written formula cannot do

1. **It cannot represent what nobody thought to write down.** Every ϕ_i exists because a human noticed the pattern and coded a detector. "The dark squares around his king are permanently weak because the bishop that guarded them was traded and the pawn structure cannot repair it" is one feature to a strong player, and roughly no feature at all to a formula — there is no counter for it.
2. **Its terms are added, so it cannot represent *it depends*.** A passed pawn on d5 is worth a great deal in one structure and nothing in another; a knight on the rim is bad, except when it is heading for f5. Equation (1.4) scores each feature in isolation and sums. Interactions have to be hand-coded as yet more special-case terms, and the number of interactions explodes faster than anyone can write them.

Compensation for sacrificed material is where both failures bite at once, which is exactly your interest in this project: a sacrifice is precisely a position where the material term is loud and wrong, and the compensating factors are diffuse, interacting, and unnamed.

Key idea

Both of Leela's departures are attacks on these limits. The network learns the features instead of being told them (fixing failure 1), and its layers multiply and gate information rather than merely summing it (fixing failure 2). Chapter 11 is precisely the story of how "it depends on what else is on the board" becomes an arithmetic operation.

1.6 What is actually in `engine/`

To keep this concrete, here is the object this book is about, as it exists on your disk.

in this repository

- `engine/lc0.exe` — the search. A C++ program, version 0.32.1. It contains no chess knowledge beyond the rules: it builds and walks a tree (Chapters 5–8).
- `engine/BT3-768x15x24h-swa-2790000.pb.gz` — a network. 15 transformer layers, 768-dimensional square embeddings, 24 attention heads (Chapters 11, 12).
- `engine/791556.pb.gz` — a smaller, faster network. All the real numbers in this book come from this one, because it runs on this machine's CPU in seconds rather than minutes.
- `engine/stockfish/` — Stockfish 16.1, used in this book only as an independent referee for chess claims.

The split matters and is worth saying once plainly: **`lc0.exe` without a network file cannot play chess at all**, and the network without `lc0.exe` plays about as well as a strong club player glancing at a board for a tenth of a second. The engine you know is the pair. Part II is the first half; Part IV is the second; Chapter 7 is where you watch them talk to each other.

Exercises

Exercise 1.1 *(warm-up)* Using $b = 31.35$ (your corpus mean), how many positions are there at depth 5? At depth 7? Roughly how many times bigger is depth 7 than depth 5, and why is that ratio not a coincidence?

Exercise 1.2 *(the hardware fallacy)* A machine evaluates 2×10^7 positions per second and searches to depth 9 in a certain time. You buy hardware 64× faster. To what depth can you now search in the same time? Show the calculation.

Exercise 1.3 *(branching factor in the endgame)* Your corpus gives $b = 32.5$ in the middlegame and $b = 15.8$ in the endgame. For a fixed budget of 10^9 leaf evaluations, how deep can exhaustive search go in each? Comment on why engine analysis "feels" so much more definite in endgames.

Exercise 1.4 *(minimax)* White to move. Root has three children A, B, C (Black to move at each). Their children's leaf values, from White's point of view, are $A : \{+1, +6, +2\}$, $B : \{+3, +3, +4\}$, $C : \{+9, -2, +7\}$. Compute $V(A), V(B), V(C)$ and the root value, and give White's best move. Which leaf is the largest number in the whole tree, and does it matter?

Exercise 1.5 *(negamax)* Redo the previous exercise using Equation (1.3), i.e. with every value expressed from the point of view of the player to move at that node. Write down each node's value in that convention and confirm you get the same best move.

Exercise 1.6 *(alpha–beta by hand)* Take the tree from Exercise 1.4, searched left to right (A then B then C; within each node, the leaves in the order listed). After finishing A, what is α? Walk through B and C and mark every leaf that alpha–beta can skip. How many of the 9 leaves did you have to look at?

Exercise 1.7 *(ordering matters)* Same tree, but now search the children in the order C, B, A. How many leaves are examined? What does the difference tell you about why classical engines invest so heavily in move-ordering heuristics — and what the learned equivalent of a move-ordering heuristic is called in Leela?

Exercise 1.8 *(writing an evaluation)* Extend the material-only $f(s)$ of the worked example with one positional feature of your own choosing (say, ϕ_6 = number of White rooks on open files minus the same for Black). Pick a weight. Now describe, in one sentence each, two different

positions where your feature gives the wrong answer, and say which of the two failure modes of §1.5.1 each one is.

Exercise 1.9 *(the compensation problem)* In one paragraph, explain why a piece sacrifice for a long-term attack is the hardest case for Equation (1.4), referring to both failure modes. Keep the chess vague and the argument sharp — this is a statement about the *form* of the equation, not about any particular sacrifice.

Exercise 1.10 *(looking ahead)* Leela examines roughly 800–2,000 positions to choose a move in the configuration this project uses, while a classical engine examines tens of millions. Before reading on, write down your best guess at how that can possibly be competitive. Keep your answer; you will check it against Chapter 6.

Chapter 2

From +1.4 to a probability: the currency of evaluation

After this chapter you can

- Say what a centipawn evaluation actually claims, and why the claim is empirical rather than definitional.
- Build the logistic curve from scratch and use it to convert an evaluation into a win probability.
- Read a WDL triple, convert between (w, d, l), expected score, and V, and do it on real Leela output.
- Explain — with arithmetic — why Monte-Carlo search requires probabilities and would be broken by centipawns.
- Distinguish two positions with identical evaluations but opposite *character*, and quantify the difference.

2.1 What does +1.4 mean?

A classical engine says +1.4. Ask what it means and you get "White is better by about a pawn and a half" — which just restates the units. The honest answer is that a centipawn score means nothing on its own; it acquires meaning only through a claim about outcomes: *positions that score* +1.4 *tend to be won by White at some particular rate.*

That is an empirical statement. Somebody has to go and measure it, and the answer depends on who is playing — +1.4 between two 1600s and +1.4 between two engines are different propositions about the future. Centipawns are a **unit of account**, not a unit of truth, and the exchange rate to reality has to be measured.

Leela cuts out the middle step. Its network does not estimate "how many pawns"; it estimates *the outcome distribution directly* — how often this position is won, drawn, lost. That single design choice is the reason its numbers can be reasoned about, averaged, and combined, and it is what makes the search of Part II mathematically legitimate.

2.2 The score of a game, and expected score

Chess has three outcomes. Score them the way tournaments do:

$$\text{win} = 1, \qquad \text{draw} = \tfrac{1}{2}, \qquad \text{loss} = 0.$$

If a position has win probability w, draw probability d and loss probability l (with $w+d+l=1$), then the **expected score** is the average points you collect:

$$\mathbb{E}[\text{score}] \;=\; 1\cdot w \;+\; \tfrac{1}{2}\cdot d \;+\; 0\cdot l \;=\; w+\tfrac{d}{2}. \tag{2.1}$$

Engines prefer a symmetric scale running from -1 (lost) through 0 (balanced) to $+1$ (won). Rescale by $V = 2\,\mathbb{E}[\text{score}] - 1$:

$$V = 2\left(w+\tfrac{d}{2}\right) - 1 = 2w + d - 1.$$

Since $d = 1-w-l$, this collapses to something you can compute in your head:

The value of a position

$$\boxed{V \;=\; w - l} \tag{2.2}$$

Draw probability does not appear. V is simply how much more likely you are to win than to lose. It ranges over $[-1,+1]$ and is stated *from the point of view of the side to move* (Chapter 1's negamax convention).

Worked example 2.1 — reading real Leela output

Leela's verbose output reports two of the three numbers: `WL` (which is exactly $V = w - l$) and `D` (which is d). Recover w and l by solving the pair $w - l = \texttt{WL}$, $w + l = 1 - \texttt{D}$:

$$w = \frac{1 - \texttt{D} + \texttt{WL}}{2}, \qquad l = \frac{1 - \texttt{D} - \texttt{WL}}{2}. \tag{2.3}$$

For the initial position, the engine in `engine/` reports $\texttt{WL} = +0.0946$, $\texttt{D} = 0.461$:

$$w = \frac{1 - 0.461 + 0.0946}{2} = \frac{0.6336}{2} = 0.3168, \qquad l = \frac{1 - 0.461 - 0.0946}{2} = \frac{0.4444}{2} = 0.2222.$$

Real engine outputtitle

Produced by `tools/collect_engine_data.py` with the network `engine/791556.pb.gz`; the Stockfish column is Stockfish 16.1 at depth 30, used as an independent referee.

Position	w	d	l	$V = w - l$	Stockfish d30
Initial position	0.317	0.461	0.222	+0.095	+0.35
Opposition draw (K+P, §2.6)	0.003	0.997	0.000	+0.003	0.00
K+P, White to move (won)	0.942	0.058	0.000	+0.942	mate in 12
Same, Black to move (lost)	0.000	0.052	0.948	−0.948	mate in 16
Morphy position before 16.Qb8+	0.713	0.193	0.094	+0.620	**mate in 2**

That last row is worth a pause, because it is a lesson the rest of the book keeps returning to. In a position with a forced mate in two on the board, this network's evaluation is +0.62 — "clearly better, probably winning". It is not lying and it is not broken. It is reporting *how positions of this kind tend to go*, which is not the same as *what is true here*. Finding the mate is the search's job (you will watch it happen, in real numbers, in Chapter 9).

2.3 Converting a centipawn score into a probability

You will constantly need to move between the two currencies, since Stockfish speaks centipawns and Leela speaks probabilities. The bridge is a curve we can build from scratch.

2.3.1 Building the logistic function

We need a function that takes an evaluation x (any real number, positive or negative) and returns a probability (between 0 and 1). Build it in three steps.

Attempt 1: a straight line

$p = \frac{1}{2} + kx$. Fails immediately: at $x = +10$ it returns a probability above 1. Any straight line eventually leaves $[0, 1]$. We need something that *saturates*.

Attempt 2: work with odds instead

Instead of probability, use **odds** $= p/(1-p)$: a probability of 0.75 is odds of 3. Odds live in $(0, \infty)$ — half the problem gone. And odds *multiply* naturally: doubling your advantage should double your odds, not add a fixed amount of probability. Take logs and the multiplication becomes addition:

$$\log\text{-odds} = \ln \frac{p}{1-p} \in (-\infty, +\infty).$$

Now *that* is a scale on which a linear model is sensible. So posit the simplest possible relationship:

$$\ln \frac{p}{1-p} \;=\; k\,x.$$

Attempt 3: solve it back for p

Exponentiate: $\frac{p}{1-p} = e^{kx}$, so $p = (1-p)e^{kx}$, so $p(1+e^{kx}) = e^{kx}$:

$$p = \frac{e^{kx}}{1+e^{kx}} = \frac{1}{1+e^{-kx}}.$$

The logistic (sigmoid) function

$$\sigma(z) \;=\; \frac{1}{1+e^{-z}}, \qquad p(\text{win-ish}) \;=\; \sigma(k\,x) \tag{2.4}$$

with x the evaluation and k a scale constant *fitted to data*, not derived. Note $\sigma(0) = 1/2$, $\sigma(+\infty) = 1$, $\sigma(-\infty) = 0$, and the symmetry $\sigma(-z) = 1 - \sigma(z)$.

You will meet σ three more times in this book, wearing different hats: as the output of a value head (Chapter 12), as the thing that turns a linear probe's raw score into a probability (Chapter 13), and as the single-neuron ancestor of the whole network (Chapter 10). It is the same function every time. Learn it once.

Worked example 2.2 — calibrating and checking the conversion

A common fitted value (used by Lichess for its evaluation bars) is expected score $= \sigma(0.00368\,\text{cp})$, with cp in centipawns.
Take Stockfish's +35 centipawns for the initial position:

$$\sigma(0.00368 \times 35) = \sigma(0.1288) = \frac{1}{1+e^{-0.1288}} = \frac{1}{1+0.8792} = 0.5321.$$

Expected score 0.532, so by Equation (2.1) rescaled, $V = 2(0.532) - 1 = +0.064$.
Leela's own answer for the same position was $V = +0.095$ (§2.2). Two engines, two architectures, two definitions — and they agree that the first move is worth about 0.06–0.10 of a game result, i.e. that White scores about 53% from the start. That is a genuine,

checkable cross-validation of the conversion, and about as much precision as the conversion deserves.

Where people go wrong

The constant k is not a law of nature. It depends on *whose* games were used to fit it: at club level, a +1.5 position is won far less reliably than between engines, so the same centipawn number maps to a different probability. Any tool that displays "White wins 78%" must be able to say 78% *of the time, between whom*. This is the first place in this book where a number can be technically correct and pedagogically dishonest, and it will not be the last.

2.4 Why the search needs probabilities

Here is the argument that makes this chapter structural rather than cosmetic. Monte-Carlo search does exactly one thing with the numbers coming back from the network: it **averages** them (Chapter 3, $Q = W/N$). So the question is whether averaging is a meaningful operation in each currency.

Worked example 2.3 — averaging in two currencies

A move leads to two equally likely continuations. In one, you emerge a queen up; in the other, you are mated.

In centipawns: the two leaves score +900 and $-\infty$ (mate). What is the average? The question is not even well-posed — engines have to fake it with a large finite number like ± 32000, and then the average of +900 and −32000 is −15550, a number that corresponds to no position and no fact. Worse, the scale is not linear in anything: being +900 is not "nine times better" than being +100, because both are usually just winning.

In probabilities: the two leaves score $V = +1$ and $V = -1$. The average is 0, and the average *means something exact*: half the futures examined are won and half are lost, so the expected score is $\frac{1}{2}$ — a coin flip. That is a true and useful summary of the position, and it is the number a player would want.

Key idea

Probabilities are *averageable*: the average of a set of win probabilities is itself a win probability, and it is the win probability you would get by picking one of those futures at random. Centipawns have no such property. Since Monte-Carlo search *is* averaging, Leela's value head is not a stylistic choice — it is the thing that makes the search coherent. An engine cannot back up centipawns through a Monte-Carlo tree without inventing meaning it does not have.

There is a caveat that costs a paragraph, because it will come back in Chapter 8. Averaging is the right operation for "what does a random future look like", but it is the *wrong* operation for "what happens with best play". If one of my two continuations is a forced mate against me, the position is lost, not a coin flip — my opponent will simply choose it. The average of +1 and −1 being 0 is exactly the minimax-versus-averaging tension, and every Monte-Carlo engine has to manage it. It is managed, not solved: the search visits the losing branch more and more, dragging the average down, so the answer converges to the truth without ever being computed as a minimum. The speed of that convergence is a real limitation of the method and one of the honest disadvantages of Leela versus alpha–beta on sharp forcing positions.

2.5 Sharpness: two positions, one evaluation

Because $V = w - l$ discards d, a single number cannot tell you what kind of fight you are in. Two positions can share $V \approx 0$ and be nothing alike:

	w	d	l	V	Character
Position A	0.05	0.90	0.05	0.00	dead level; hard to lose, hard to win
Position B	0.45	0.10	0.45	0.00	mutual minefield; someone is going to be wrong

(Illustrative numbers.) These demand opposite treatment from a training tool, and opposite treatment from you at the board. A single evaluation bar cannot distinguish them. The WDL triple can, and this is the cheapest, most under-used signal Leela produces.

Define a **sharpness** measure as the probability that the game is *decided*:

$$\text{sharp}(s) \;=\; w + l \;=\; 1 - d. \tag{2.5}$$

Position A scores 0.10; position B scores 0.90. Same V, ninefold difference in what is at stake. And it is not a hand-waving construction — the engine really does report it:

Real engine outputtitle

Position	d	sharp $= 1 - d$	
Opposition draw (K+P)	0.997	0.003	nothing can happen
Initial position	0.461	0.539	a normal game
Morphy position (before Qb8+)	0.193	0.807	something is about to be decided
K+P, White to move	0.058	0.942	already decided

Note that the last two rows have very different V (+0.62 vs +0.94) but similar sharpness — and the first two have similar V (both near 0) and utterly different sharpness. The two numbers are genuinely independent axes.

Key idea

V tells you *who* stands better. $1 - d$ tells you *whether it matters yet.* Your project's interest in "sharp, dynamic chess" is, in this vocabulary, an interest in positions with high $1 - d$ — and Chapter 16 shows that you can steer an engine towards them by writing that preference into an objective function.

2.5.1 What Leela's other columns mean

While we are here, the rest of a verbose line, so nothing in this book is mysterious later:

`V`	the network's raw evaluation of this move's position: one forward pass, no search
`WL`	$w - l$ of the current node, i.e. V after search
`D`	draw probability d
`M`	predicted number of plies remaining in the game (a third head; see ch. 12)
`N`	visits (Chapter 3)
`P`	policy prior (Chapter 6)
`Q`	running average of everything backed up through this move
`U`	the exploration bonus (Chapter 6)
`S`	$Q + U$: the PUCT score that actually decides where to look next

By Chapter 7 you will be able to compute `U` and `S` yourself from the other columns, and check the engine's arithmetic. You will find it checks out to five decimal places.

in this repository

The WDL triple is already plumbed through this project: the frontend's evaluation display and the "character of the fight" language in the design documents both come from d. The sharpness quantity $1 - d$ of Equation (2.5) is the simplest ancestor of the tactical-steering score you build in Chapter 17.

Exercises

Exercise 2.1 *(conversions)* A position has $w = 0.55$, $d = 0.30$, $l = 0.15$. Compute the expected score and V. Another has $w = 0.40$, $d = 0.60$, $l = 0.00$. Which has the larger V? Which would you rather have, and why might the answer depend on the tournament situation rather than the position?

Exercise 2.2 *(recovering WDL)* Leela reports $\texttt{WL} = -0.312$, $\texttt{D} = 0.244$. Recover w, d, l with Equation (2.3) and check they sum to 1. Whose point of view are these from, and what changes if it is Black to move?

Exercise 2.3 *(the logistic by hand)* Compute $\sigma(z)$ for $z = -2, -1, 0, 1, 2$. Sketch the curve. Where is it steepest, and what does that steepness say about which positions an extra tenth of a pawn matters most in?

Exercise 2.4 *(centipawns to probability)* Using $\sigma(0.00368\,\text{cp})$, convert +50, +150, +300 and +800 centipawns into expected scores. Compute the difference in expected score between +50 and +150, and between +700 and +800. Both are a 100-centipawn gain — explain in one sentence why they are worth wildly different amounts.

Exercise 2.5 *(the ratings problem)* k in Equation (2.4) is fitted to a population. Should k be larger or smaller when fitting to 1200-rated players than to engines? Draw both curves on the same sketch, and say what this implies for a training tool that shows you a win probability.

Exercise 2.6 *(averaging)* A move has four sampled continuations with $V = +0.9, +0.8, +0.85, -1.0$. Compute the average. Now suppose the -1.0 is a forced mate against you that the opponent can choose. What is the position actually worth? Explain the discrepancy in terms of §2.4, and say what the search must do to fix it.

Exercise 2.7 *(sharpness)* Two positions: $(w, d, l) = (0.30, 0.55, 0.15)$ and $(0.30, 0.20, 0.50)$. Compute V and $1 - d$ for both. Write the one-sentence description a coach should give of each, using no chess terms at all.

Exercise 2.8 *(designing a display)* You must show a position's evaluation in your tool using at most two visual elements. Argue for a specific choice of the two quantities to display, given a user who wants sharp, dynamic play. What would be lost by showing the classical single bar?

Exercise 2.9 *(the mate that reads as +0.62)* In §2.2, a position with mate in two was evaluated by the network at $V = +0.62$. Explain why this is not a bug, using the word "population" in your answer. Then say what would have to change for the number to become +1.0, and which chapter of this book is about that change.

Exercise 2.10 *(a calibration experiment)* Design (do not run) an experiment on your 9,030-game corpus that would measure the true k for *your* games: what would you compute, what would you plot, and what would the plot look like if the standard k were too large? Be specific about the buckets.

Part II

Searching by Gambling Well

Chapter 3

One slot machine: estimating a value you can only sample

After this chapter you can

- Explain why a chess move's value is something you *sample* rather than compute.
- Derive the running-average update $Q \leftarrow Q + (x - Q)/n$ from the definition of a mean, and recognise it as Leela's W/N.
- Say how wrong a sample mean can be after n samples, and why the answer shrinks like $1/\sqrt{n}$ and not like $1/n$.
- Turn "how wrong could I be?" into an explicit $\pm$ number using Hoeffding's bound, and thereby produce the exact expression $\sqrt{\ln(\cdot)/n}$ that will become the exploration bonus in Chapter 4.

3.1 Why a move's value has to be sampled

In Chapter 1 we treated evaluation as a function you *compute*: hand a position to f, receive a number. Monte-Carlo search takes a different view, and it is worth getting the shift straight before any mathematics.

Suppose you are considering 1. Nf5. What is it worth? Its true value is the result of perfect play from there — unavailable. What you can do instead is *look at one continuation* and see how it turns out. Then another. Then another. Each look gives you a number, and the numbers disagree, because different continuations lead to different places.

Key idea

The value of a move is not a quantity you read off. It is the **average outcome over the continuations you examine**. Each visit to a move is one *sample* of its value. The engine's job is to decide which move to sample next, and its estimate of a move is the running average of the samples it has taken.

This is the whole reason a casino metaphor shows up in a chess book. A slot machine is a device that returns a random payout each time you pull it; you can only learn what a machine is worth by pulling it. A candidate move, in Monte-Carlo search, behaves exactly like that: pull it (search one line through it), get a payout (the evaluation at the end of the line), and update your opinion.

Two warnings, now rather than later.

Where people go wrong

(1) Leela does not play random games. In the original Monte-Carlo tree search of the 2000s, a "sample" meant playing the game out to the end with random moves and scoring the result. Leela never does this. Its sample is a single network evaluation of the position at the end of the line — one forward pass, no rollout. The *statistics* are Monte-Carlo; the *samples* are not random games. Chapter 5 makes this precise.

(2) The samples are not independent, and get less random over time. Real slot machines pay out independently; a chess move's samples are chosen deliberately and become increasingly concentrated on the best line. That is a feature, but it means the classical guarantees in this chapter are motivation for the formula, not proof about the engine. Keep that distinction; Chapter 4 returns to it.

3.2 The running average, and where W/N comes from

Say a move has been visited n times, producing values $x_1, x_2, \ldots, x_n$. Its estimate is the sample mean, which we will call Q from the start, because that is its name in every engine log you will ever read:

$$Q_n \;=\; \frac{1}{n}\sum_{i=1}^{n} x_i \;=\; \frac{W_n}{n}, \qquad \text{where } W_n = \sum_{i=1}^{n} x_i. \tag{3.1}$$

That is already the engine's data structure. Leela stores, on every edge of its tree, exactly two numbers: W (the running total of everything backed up through this edge) and N (how many times it has been visited). $Q = W/N$ is not a concept, it is a division.

3.2.1 Updating without re-adding everything

You do not want to keep the whole list $x_1, \ldots, x_n$. You want to update Q in place. Start from the definition and do nothing clever:

$$Q_n = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{1}{n}\left(\sum_{i=1}^{n-1} x_i + x_n\right) = \frac{1}{n}\Big((n-1)\,Q_{n-1} + x_n\Big).$$

Expand and regroup:

$$Q_n = \frac{n\,Q_{n-1} - Q_{n-1} + x_n}{n} = Q_{n-1} + \frac{x_n - Q_{n-1}}{n}.$$

The incremental mean

$$Q_n \;=\; Q_{n-1} \;+\; \frac{1}{n}\,(x_n - Q_{n-1}). \tag{3.2}$$

New estimate = old estimate + (surprise) × (a shrinking step size). The quantity $x_n - Q_{n-1}$ is how much the newest sample disagreed with what you believed; $1/n$ says a disagreement counts for less and less as evidence accumulates.

Worked example 3.1 — watching an estimate settle

A move is sampled six times, giving $x = 0.90,\ 0.20,\ 0.75,\ 0.85,\ 0.60,\ 0.80$. (Illustrative numbers, not engine output.) Apply Equation (3.2):

n	x_n	surprise $x_n - Q_{n-1}$	step $\frac{1}{n}$	Q_n
1	0.90	—	—	0.900
2	0.20	$0.20 - 0.900 = -0.700$	0.500	$0.900 - 0.350 = 0.550$
3	0.75	$0.75 - 0.550 = +0.200$	0.333	$0.550 + 0.067 = 0.617$
4	0.85	$0.85 - 0.617 = +0.233$	0.250	$0.617 + 0.058 = 0.675$
5	0.60	$0.60 - 0.675 = -0.075$	0.200	$0.675 - 0.015 = 0.660$
6	0.80	$0.80 - 0.660 = +0.140$	0.167	$0.660 + 0.023 = 0.683$

Check: $W_6 = 0.90 + 0.20 + 0.75 + 0.85 + 0.60 + 0.80 = 4.10$, and $4.10/6 = 0.683$. ✓
Two things to notice, both of which have chess consequences.

- **The second sample moved the estimate by 0.35; the sixth moved it by 0.02.** Early samples dominate. In a search tree, this is why a move whose first look is terrible can be abandoned quickly — and why an engine at 50 nodes has genuinely flimsy opinions.
- **One refutation drags an average down but does not zero it.** Q after the disastrous 0.20 was still 0.55. An average is a compromise between what you have seen; chess is not a compromise — if one reply refutes the move, the move is refuted. This is a real weakness of averaging, and Chapter 8 covers what engines do about it.

3.3 How wrong is a sample mean?

Q_n is an estimate of the move's true value μ. The engine must know not just its estimate but *how much to trust it*, because that is what tells it whether to look again. So: after n samples, how far can Q_n be from μ?

3.3.1 The $1/\sqrt{n}$ law

Here is the fact that governs everything (stated; the proof is standard and not needed):

Confidence: established

If the samples are independent draws with true mean μ and standard deviation σ, then the sample mean Q_n has standard deviation

$$(\text{typical error of } Q_n) \;=\; \frac{\sigma}{\sqrt{n}}. \tag{3.3}$$

The square root is the single most consequential fact in this book, so make it concrete. Take $\sigma = 0.4$ (plausible for values in $[-1, +1]$):

Samples n	Typical error $0.4/\sqrt{n}$	
1	0.400	the estimate is nearly worthless
4	0.200	
16	0.100	
64	0.050	
256	0.025	
1024	0.013	

Key idea

To halve your uncertainty you must quadruple your evidence. Precision is bought at quadratically increasing cost. This is why a search cannot afford to be precise about everything: driving one move's error from 0.10 to 0.05 costs the same 48 extra visits that could have given four other moves their first look. *Precision is a budget, and spending it is the search's entire job.*

There is a direct chess reading of this. When an engine reports an evaluation after a short search, the last digit is noise. When your tool tells you a move loses 0.08 of evaluation, Equation (3.3) is the reason you should ask how many nodes were behind that number before you believe it. Chapter 9 turns this into a practical rule.

3.3.2 From "typical error" to a guarantee

"Typical error" is not enough for an algorithm. An algorithm needs a statement of the form *"I am confident the truth is not more than ϵ above my estimate."* That is a one-sided bound, and it is exactly what the search will use to decide whether a move might still be better than it looks.

Hoeffding's inequality provides one. For samples bounded in an interval of width R (for us $x_i \in [-1, +1]$, so $R = 2$):

Confidence: established

Hoeffding's inequality. For independent samples in an interval of width R,

$$\Pr(\,\mu \geq Q_n + \epsilon\,) \;\leq\; \exp\!\left(-\frac{2n\epsilon^2}{R^2}\right). \tag{3.4}$$

In words: the chance that the truth exceeds your estimate by ϵ or more falls off *exponentially* in $n\epsilon^2$.

Now invert it. Suppose we are willing to be wrong with probability at most δ. Set the right-hand side equal to δ and solve for ϵ:

$$\exp\!\left(-\frac{2n\epsilon^2}{R^2}\right) = \delta \implies -\frac{2n\epsilon^2}{R^2} = \ln\delta \implies \epsilon^2 = \frac{R^2\ln(1/\delta)}{2n},$$

$$\boxed{\epsilon \;=\; R\sqrt{\frac{\ln(1/\delta)}{2n}}} \tag{3.5}$$

Stop and look at the shape of Equation (3.5), because you are going to meet it again in six pages wearing a different hat:

- ϵ shrinks like $1/\sqrt{n}$ — the same square root as before;
- it grows like $\sqrt{\ln(1/\delta)}$ — demanding more certainty costs you, but only *logarithmically*, which is cheap;
- it is proportional to the range R — noisier quantities need wider bounds.

Worked example 3.2 — a concrete confidence bound

A move has been visited $n = 25$ times with $Q = 0.30$; values lie in $[-1, +1]$ so $R = 2$. We want a bound that is wrong at most 5% of the time, $\delta = 0.05$.

$$\epsilon = 2\sqrt{\frac{\ln(1/0.05)}{2\times 25}} = 2\sqrt{\frac{\ln 20}{50}} = 2\sqrt{\frac{2.996}{50}} = 2\sqrt{0.0599} = 2\times 0.2448 = 0.490.$$

So we can say, with 95% confidence, $\mu \le 0.30 + 0.49 = 0.79$.
That is a very loose statement — after 25 visits the move could plausibly be nearly winning or clearly worse than we think. Now the same computation at $n = 400$:

$$\epsilon = 2\sqrt{\frac{2.996}{800}} = 2 \times 0.0612 = 0.122, \qquad \mu \le 0.42.$$

Sixteen times the work, four times tighter. The $1/\sqrt{n}$ law, in the flesh.

3.4 The optimism principle

We now have, for any move, two numbers: what it has scored (Q_n) and how much room there is above that (ϵ). The quantity

$$\underbrace{Q_n}_{\text{measured}} + \underbrace{\epsilon}_{\text{room for error}} \tag{3.6}$$

is called an **upper confidence bound**: the best this move could plausibly be, given what you have seen so far. It is the highest value you cannot yet rule out.

Key idea

Optimism in the face of uncertainty. When choosing what to examine next, rank moves not by what they have scored, but by *the best they could plausibly be*. Then one of two good things happens on every visit: either the move really is that good — excellent, you found it — or it is not, and the visit shrinks ϵ, so the illusion cannot persist. Either way the visit was not wasted.

That single sentence is the engine's entire attitude to thinking. Everything from here to Chapter 7 is a matter of writing down ϵ well: Chapter 4 turns Equation (3.6) into UCB1 by choosing δ sensibly, Chapter 5 puts it on a tree, and Chapter 6 replaces the statistical ϵ with a *learned* one — which is where a chess engine's intuition finally enters the mathematics.

in this repository

W and N per edge are the whole state of Leela's tree. In the verbose output you already have (`-verbose-move-stats`), the fields `N:` and `Q:` are exactly n and Q_n of this chapter; `U:` is the ϵ of Equation (3.6), built in Chapter 6. There is no W column because $W = Q \cdot N$.

Exercises

Exercise 3.1 *(running average)* A move is visited five times with values 0.10, 0.60, 0.40, 0.50, 0.90. Use Equation (3.2) step by step to get Q_5, showing the surprise and the step size each time. Verify against W/N.

Exercise 3.2 *(the weight of the first look)* Using Equation (3.2), by how much can a single sample move the estimate when $n = 2$? When $n = 50$? A move currently has $Q = 0.80$ over $N = 50$ visits and its next visit returns -1.0 (it loses on the spot). What is the new Q? Comment on what this means for an engine trying to notice a refutation that only appears deep in a well-trodden line.

Exercise 3.3 *(the square-root law)* With $\sigma = 0.4$, how many samples do you need for a typical error below 0.02? Below 0.01? What is the ratio of those two answers, and what general rule does it illustrate?

Exercise 3.4 *(budget arithmetic)* You have 100 visits and two moves. Option A: 50 visits each. Option B: 90 to the first, 10 to the second. Using $\sigma = 0.4$, compute the typical error on each move under both plans. Which plan is better if you must *rank* the two moves, and which is better if you must *report an accurate evaluation* for the first one? (There is no single right answer; the point is that the right split depends on the question being asked.)

Exercise 3.5 *(Hoeffding)* Compute the 95% upper bound ϵ from Equation (3.5) for $n = 1, 4, 16, 64, 256$ with $R = 2$. Tabulate. At what n does the bound first become narrower than the whole range of a decisive-versus-drawn distinction (say $\epsilon < 0.25$)?

Exercise 3.6 *(the price of certainty)* Compare ϵ at $\delta = 0.05$ and at $\delta = 0.001$ for fixed $n = 100$, $R = 2$. By what factor did the bound widen when you demanded 50 times more certainty? Which term in Equation (3.5) is responsible, and why is that reassuring for an algorithm that must be right many times in a row?

Exercise 3.7 *(optimism, by hand)* Three moves have (Q, n) of $A : (0.55, 100)$, $B : (0.40, 4)$, $C : (0.62, 900)$. Using $R = 2$, $\delta = 0.1$, compute $Q + \epsilon$ for each. Which move does the optimism principle examine next? Which move would a greedy "pick the best average" rule examine? State in one sentence what the optimism rule is buying with the difference.

Exercise 3.8 *(where the metaphor breaks)* Give two specific ways a chess move differs from a slot machine, beyond those mentioned in this chapter. For each, say whether it makes the estimate Q too optimistic, too pessimistic, or merely noisier.

Exercise 3.9 *(reading the log)* An engine line reads `N: 4 ... (Q: 0.61)`. Another reads `N: 1900 ... (Q: 0.58)`. A user concludes "the first move is better". Write the two-sentence correction you would put in a training tool, using a number from this chapter.

Chapter 4

Many machines: exploration, regret, and the first bonus term

After this chapter you can

- Define regret, and use it to say precisely what "wasted thinking" means.
- Show by explicit counterexample that always picking the best-so-far move is a catastrophic policy, and that adding randomness is a poor repair.
- Build the UCB1 rule from Chapter 3's confidence bound, through three failed attempts, and understand why the final bonus contains $\ln N$ over n_i under a square root.
- Run UCB1 by hand for twelve pulls over three machines and watch it self-correct.
- State what UCB1 guarantees — and where that guarantee stops applying to chess.

4.1 The setup, and the honest definition of waste

You face K machines. Machine i has an unknown true value μ_i; pulling it returns a noisy sample. You have T pulls in total. In chess: K is the number of legal moves at the node (≈ 31, Chapter 1), a pull is one visit, and T is the node budget — 800, or 1,600, or whatever the tool allows.

How do we score a strategy? Not by "did it find the best machine", which is too crude. The standard measure is **regret**: how much worse you did than a genie who knew μ from the start and pulled the best machine every time.

$$\text{Regret}(T) \;=\; \underbrace{T\,\mu^*}_{\text{the genie's total}} \;-\; \underbrace{\sum_{t=1}^{T} \mu_{i_t}}_{\text{your total}}\,, \qquad \mu^* = \max_i \mu_i. \tag{4.1}$$

Equivalently, if machine i is worse than the best by $\Delta_i = \mu^* - \mu_i$ and you pulled it n_i times, then

$$\text{Regret}(T) \;=\; \sum_i n_i \Delta_i. \tag{4.2}$$

Equation (4.2) is the one to remember, because it says something a chess player recognises: **a bad move is not expensive to look at if it is only slightly bad, and looking briefly at a terrible move costs almost nothing — what ruins you is spending many visits on something clearly inferior.** A search that spends 700 of its 800 nodes on a move that loses is not "thorough", it is a search with enormous regret.

Notation

K machines/moves; μ_i true value of i; μ^* the best; $\Delta_i = \mu^* - \mu_i$ its *gap*; n_i times pulled; $N = \sum_i n_i$ total pulls so far; Q_i the running average of machine i (Chapter 3).

4.2 Attempt 0: just pick the best so far (greedy)

The obvious rule: pull $\arg\max_i Q_i$.

Worked example 4.1 — how greedy destroys itself

Two machines. Truth (unknown to the algorithm): $\mu_A = 0.9$, $\mu_B = 0.5$. Samples are noisy, so any single pull can mislead.

1. Pull each once. A has an unlucky sample: $x_A = 0.1$. B has a fair one: $x_B = 0.5$. Now $Q_A = 0.1$, $Q_B = 0.5$.
2. Greedy pulls $\arg\max Q = B$ forever. Every B sample keeps Q_B near $0.5 > 0.1$. A is never touched again, so Q_A *never changes.*

Over $T = 800$ pulls the regret is roughly $798 \times (0.9 - 0.5) = 319$: essentially the maximum possible. And note the mechanism — **the algorithm has no way of ever discovering its error**, because the only thing that could correct it is the very action it has stopped taking.

That failure has a precise chess name. A single shallow look at a strong move can return a bad number (the refutation of the refutation is two plies further on). If the search then never returns to that move, the engine has locked in a blunder *and cannot detect it.* Any usable search must guarantee that a mistakenly-dismissed move gets another chance.

4.2.1 Attempt 0b: greedy with a coin (ϵ-greedy)

Standard first repair: with probability ϵ (say 0.1) pull a machine at random, otherwise be greedy. This does fix the fatal flaw — A will eventually be re-sampled — but it is crude in two ways that matter for chess.

1. **It never stops wasting.** It keeps spending 10% of the budget uniformly at random forever, including on machines already known to be terrible. In chess, with $K \approx 31$, that means a permanent tax paid mostly on moves that hang a piece.
2. **It explores blindly.** It cannot distinguish "unpromising and well-measured" from "unpromising and barely looked at". Those deserve completely different treatment, and ϵ-greedy treats them identically.

We already have the tool that fixes both: the confidence bound of Chapter 3. Exploration should be aimed at *uncertainty*, not sprayed.

4.3 Building the bonus, one failure at a time

We want a rule of the form

$$\text{score}_i \;=\; Q_i \;+\; (\text{bonus}_i), \tag{4.3}$$

pull the highest score, where the bonus expresses "how much room there might be above what I have measured". Chapter 3 says the room is $\epsilon = R\sqrt{\ln(1/\delta)/2n_i}$. The only real question is what to put in place of δ. Let us get there by failing three times.

Attempt 1: a constant bonus

$$\text{score}_i = Q_i + c.$$

Useless: adding the same constant to everyone changes nothing about the ranking. The bonus must depend on how much you have looked at each machine.

Attempt 2: bonus $= c/n_i$

Now the bonus shrinks with evidence, which is the right direction. But it shrinks *too fast*, and something worse: it is unaffected by the passage of time.
Concretely, with $c = 1$: a machine pulled 10 times has bonus 0.1. Suppose its Q is 0.30 while the current leader sits at $Q = 0.55$. Its score is 0.40, it never wins the argmax — and as the leader is pulled another thousand times, *nothing about our machine changes*. Its bonus stays 0.1 forever.
Why is that wrong? Because the meaning of "unexplored" is relative. A machine with 10 pulls is well understood when the total budget spent is 20, and barely touched when the total is 100,000. **The bonus must grow as the rest of the search grows.**

Attempt 3: bonus $= c/\sqrt{n_i}$

Better — this is the honest $1/\sqrt{n}$ shape of Equation (3.3) — but it has exactly the same defect: no dependence on total pulls N. A machine can be permanently frozen out by a rival that got lucky early, no matter how long the search runs.
We need a numerator that grows with N.

4.3.1 Choosing δ: how the $\ln N$ appears

Return to Equation (3.5): $\epsilon = R\sqrt{\ln(1/\delta)/2n_i}$. We must choose the failure probability δ. Here is the reasoning that fixes it.

We are not making one statement; we are making one statement *per machine, per time step*, and we would like all of them to hold simultaneously. If each individual statement may fail with probability δ, then over N time steps the chance that *some* statement fails grows with N. So δ must shrink as N grows. The standard choice (Auer, Cesa-Bianchi and Fischer, 2002) is

$$\delta = N^{-4}, \qquad \text{giving} \qquad \ln(1/\delta) = 4\ln N.$$

Substituting into Equation (3.5) with R and the constants folded into a single tunable c:

$$\epsilon_i = R\sqrt{\frac{4\ln N}{2n_i}} = \underbrace{R\sqrt{2}}_{\text{call it } c}\sqrt{\frac{\ln N}{n_i}}.$$

UCB1

$$\boxed{\text{pull} \quad \arg\max_i \left[Q_i + c\sqrt{\frac{\ln N}{n_i}} \right]} \tag{4.4}$$

with N the total pulls so far, n_i the pulls of machine i, and $c > 0$ an exploration constant. Any machine with $n_i = 0$ is treated as having infinite bonus, so every machine is pulled once before any is pulled twice.

Now read the two moving parts against the failures they repair:

- $\frac{1}{\sqrt{n_i}}$ — *"I have looked at this n_i times, so my error bar is this wide."* Repairs Attempt 1. It is the law of Chapter 3, unchanged.
- $\sqrt{\ln N}$ — *"...but the search has moved on, and being unexamined is more suspicious now than it was."* Repairs Attempts 2 and 3. It grows without bound, so no machine is frozen out forever; and it grows only *logarithmically*, so the revival is slow and cheap. Doubling the search multiplies this term by $\sqrt{\ln 2N / \ln N}$ — a few percent, not a factor.

Key idea

The bonus is a **plausibility of being underrated**. It is large when a machine is poorly measured relative to how much total measuring has gone on, and it decays as you look. The rule is not "explore sometimes"; it is "always take the option that could be best, given what you actually know" — and that single rule produces exploration as a consequence, targeted exactly where the ignorance is.

4.4 UCB1 by hand

Worked example 4.2 — twelve pulls, three machines

Truth (hidden from the algorithm): $\mu_A = 0.7$, $\mu_B = 0.5$, $\mu_C = 0.2$. To keep the arithmetic clean, assume each pull of a machine returns exactly its true value plus the listed noise. Take $c = 1$. *Illustrative numbers, not engine output.*

Pulls 1–3 (each machine once, since $n_i = 0$ gives infinite bonus): $x_A = 0.3$ (unlucky), $x_B = 0.5$, $x_C = 0.2$. So $Q_A = 0.3$, $Q_B = 0.5$, $Q_C = 0.2$, all $n_i = 1$, and $N = 3$.

Pull 4: $\ln N = \ln 3 = 1.099$, so every bonus is $\sqrt{1.099/1} = 1.048$.

$$\text{scores: } A = 1.348, \quad B = 1.548, \quad C = 1.248 \;\Rightarrow\; \text{pull } B.$$

Say $x_B = 0.5$, so $Q_B = 0.5$, $n_B = 2$, $N = 4$.

Pull 5: $\ln 4 = 1.386$.

$$A : 0.3 + \sqrt{1.386/1} = 0.3 + 1.177 = 1.477, \quad B : 0.5 + \sqrt{1.386/2} = 0.5 + 0.833 = 1.333, \quad C : 0.2 + 1.177 = 1.377.$$

$\Rightarrow$ pull A — **note what just happened**: A has the *worst* average of the two leaders, and it gets pulled anyway, because B's bonus shrank when B was measured. This is the greedy catastrophe being averted, on schedule and without randomness. Say $x_A = 0.9$, so $Q_A = (0.3 + 0.9)/2 = 0.6$, $n_A = 2$, $N = 5$.

Pull 6: $\ln 5 = 1.609$; bonuses $\sqrt{1.609/2} = 0.897$ for A and B, $\sqrt{1.609/1} = 1.269$ for C.

$$A : 0.6 + 0.897 = 1.497, \quad B : 0.5 + 0.897 = 1.397, \quad C : 0.2 + 1.269 = 1.469 \;\Rightarrow\; \text{pull } A.$$

Say $x_A = 0.7$: $Q_A = 0.733$, $n_A = 3$, $N = 6$.

Continuing the same arithmetic to $N = 12$ gives roughly $n_A = 6$, $n_B = 4$, $n_C = 2$ (exact counts depend on the noise you assume; Exercise 4.6 asks you to finish it). The shape is the thing:

Machine	true μ	pulls n_i	share of budget
A	0.7	6	50%
B	0.5	4	33%
C	0.2	2	17%

The budget is *graded*, not switched. The worst machine is not banned — it keeps a trickle — and the best gets the most without ever being declared the winner. And regret by Equation (4.2) is $4(0.2) + 2(0.5) = 1.8$, against the greedy disaster of the earlier example.

Confidence: established

The guarantee (Auer et al., 2002). UCB1's expected regret after T pulls is $O\left(\sum_{i:\Delta_i>0} \frac{\ln T}{\Delta_i}\right)$: it grows only *logarithmically* in T. Since a strategy that pulls any suboptimal machine a constant fraction of the time has regret growing *linearly*, this is a genuine and strong result: in the long run UCB1 spends a vanishing fraction of its budget on inferior options, without ever needing to know μ.

Notice also what the $1/\Delta_i$ says: machines that are *nearly* as good as the best get pulled a lot. That is not a bug. If two moves are within 0.02 of each other it does not much matter which you play, and the search correctly declines to spend its budget separating them — while a move that is 0.9 worse is dismissed in a handful of visits. Engines look sharp-eyed about blunders and indecisive about near-equal options for exactly this reason.

4.5 The exploration constant

c is the one free knob, and it does what you would expect:

$c = 0$	pure greed; the catastrophe of §4.2
c small	narrow, deep, confident search; risks locking onto an early mistake
c large	broad, shallow search; every move gets attention, nothing is resolved
$c \to \infty$	uniform sampling; you have re-invented brute force

Real engine outputtitle

Leela's equivalent constant is exposed as the UCI option `CPuct`, and on the engine in `engine/` its default value is

$$\texttt{CPuct} = 1.745,$$

alongside `CPuctBase` $= 38739$ and `CPuctFactor` $= 3.894$, which make it grow slowly with the size of the search (Chapter 6 explains why). Verified by running `lc0.exe` with the `uci` command and reading the option list.

4.6 Where the guarantee stops

Be clear about what has and has not been established, because this is exactly the sort of place where a confident-sounding book does damage.

Where people go wrong

The regret theorem assumes independent, identically distributed samples from a fixed distribution. In a chess search:

- samples are *not* independent — successive visits to a move deliberately go down different, related lines;
- the distribution is *not* fixed — as the subtree below a move grows, the value coming back changes systematically (that is the entire point of searching deeper);

- and the goal is different: we do not want to maximise the sum of payouts along the way, we want to *identify the best move at the end*. Those are different objectives and are known to have different optimal strategies.

So UCB1 is the **motivation** for what engines do, not a proof that it is optimal for them. What survives contact with practice is the *shape*: measured value plus a bonus that shrinks with evidence and grows with total effort. Every strong Monte-Carlo engine uses that shape; none of them satisfies the theorem's hypotheses.

That honest caveat also opens the door to the next two chapters. Because the theorem does not bind us, we are free to change the bonus into something better suited to chess — and in Chapter 6 we will replace the purely statistical "how much have I looked?" with "how much have I looked, *and how good does a strong player think this move is?*"

Exercises

Exercise 4.1 *(regret)* Three moves have true values $0.8, 0.75, 0.1$. A search of 800 visits allocates them $600, 100, 100$. Compute the regret using Equation (4.2). Now compute the regret for the allocation $400, 390, 10$. Which search would you rather have, and does regret agree with you?

Exercise 4.2 *(greedy)* Construct a two-machine example in which greedy achieves *zero* regret, and one in which it achieves the maximum possible. What property of the first sample decides which of these happens? What does this tell you about relying on a search's first impression of a move?

Exercise 4.3 *(ϵ-greedy's tax)* With $K = 31$ moves, $\epsilon = 0.1$, and $T = 800$ visits, how many visits go to uniformly random moves? If 25 of the 31 moves are blunders with $\Delta_i \approx 1.0$, roughly how much regret does the random component alone generate? Compare with the UCB1 behaviour described in §4.4.

Exercise 4.4 *(why $\ln N$ and not N)* Suppose someone proposes bonus $= c\sqrt{N/n_i}$ (no logarithm). Take a machine with $n_i = 1$ and $Q_i = -0.9$ and a leader with $Q = 0.8$, $n = 100$, and compute both scores at $N = 100$ and at $N = 10{,}000$. What goes wrong, and what does the logarithm buy?

Exercise 4.5 *(bonus table)* Tabulate the UCB1 bonus $\sqrt{\ln N/n_i}$ with $c = 1$ for $n_i \in \{1, 4, 16, 64\}$ at $N = 100$ and again at $N = 10{,}000$. By what factor did each bonus grow when the search got 100 times bigger? State the general rule in one sentence.

Exercise 4.6 *(finish the simulation)* Continue the worked example from pull 7 to pull 12, assuming every pull of A returns 0.7, of B returns 0.5, of C returns 0.2 exactly. Give the full table of Q_i, n_i and scores at each step, and the final visit distribution.

Exercise 4.7 *(the near-equal case)* Two moves have $\mu_A = 0.60$ and $\mu_B = 0.58$. Using the $\ln T/\Delta_i$ shape of the regret bound, argue qualitatively how the visits split, and explain why an engine that "cannot make up its mind" between two moves is usually telling you something true about the position rather than failing.

Exercise 4.8 *(budget and c)* You run a search with 200 nodes and one with 20,000 nodes. Should c be the same in both? Argue both sides, then look at the real Leela defaults in §4.6 and say which side the engine's designers took (`CPuctBase` and `CPuctFactor` are the clue; you will confirm your reading in Chapter 6).

Exercise 4.9 *(applying it to chess, carefully)* List three specific ways in which visiting a chess move differs from pulling a slot machine. For each, say whether it makes UCB1's assumption more or less reasonable, and name (guessing is fine) what an engine might do to compensate.

Exercise 4.10 *(the diagnostic)* Your tool reports that in one of your games, the engine spent 92% of its visits on a single move. In another position it spread visits 30/28/25/17 across four moves. Using this chapter's vocabulary only — no chess — describe what each distribution says about the position. (Chapter 9 turns your answer into a feature.)

Chapter 5

From machines to trees: UCT and the sign that flips

After this chapter you can

- Turn a single bandit problem into a tree of bandit problems, and state the four phases of a Monte-Carlo tree search iteration.
- Perform backpropagation with correct sign flips, by hand, and explain why Q in a Black-to-move node being $+0.9$ means Black is winning.
- Explain why Leela evaluates a leaf with one network call instead of playing the game out at random, and what was wrong with the random version.
- Compute why UCT without prior knowledge cannot function at chess's branching factor — the argument that forces the next chapter.

5.1 Bandits all the way down

Chapter 4 solved one decision: given K options with unknown values, how do you spend T pulls? A chess position is exactly that decision — and so is every position it leads to.

Key idea

A search tree is a bandit problem at every node. At the root you must choose among your legal moves. Whichever you choose, you arrive at a position where your *opponent* faces the same kind of choice, and so on. UCT (Kocsis & Szepesvári, 2006) is the observation that you can run UCB1 at every node simultaneously, and that the whole thing coheres: the value estimate at a node is fed by the estimates below it.

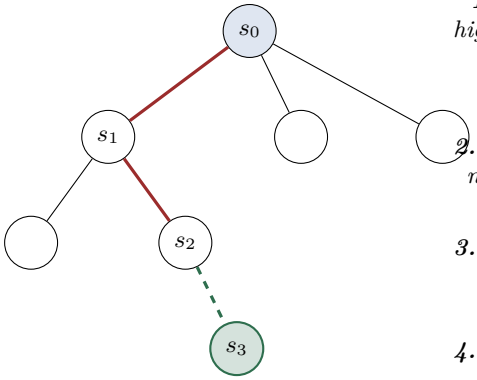
The vocabulary, fixed once:

Notation

A **node** is a position the search has stored. An **edge** is a legal move out of a stored position. Each edge carries $N(s,a)$ (visits) and $W(s,a)$ (accumulated value), with $Q(s,a) = W(s,a)/N(s,a)$. A **leaf** is a node in the tree with no children yet. The **root** is the position we must move in. $N(s) = \sum_b N(s,b)$ is the node's total visits.

5.2 One iteration, four phases

Every Monte-Carlo tree search — Leela's included — repeats the same four-phase cycle until the budget runs out. One cycle adds exactly one new node to the tree.



1. Selection. *From the root, repeatedly take the highest-scoring edge $(Q + U)$ until you reach a node whose chosen move has no child yet.*

2. Expansion. *Create that child, s_3. Ask the network for its policy over s_3's legal moves.*

3. Evaluation. *The same network call returns $V(s_3)$ — one forward pass, no rollout.*

4. Backpropagation. *Walk back up the red path, adding $V(s_3)$ to every W and 1 to every N,* flipping the sign at each ply.

The remarkable thing about this loop is how little it contains. There is no chess in it whatsoever except the move generator. All the chess lives in the network call in phase 3; the loop's only job is deciding *which* position gets that call next. This is why Chapter 1 insisted that `lc0.exe` has no chess knowledge — it is literally this loop, plus engineering.

5.2.1 Phase 3, and what Monte-Carlo used to mean

The name "Monte-Carlo" is a historical fossil, and it misleads people constantly.

In the original method (Go programs of the mid-2000s), phase 3 was a **rollout**: from the new leaf, play random legal moves until the game ends, and score the result $+1$, 0 or -1. That is where the randomness — and the casino name — came from. Averaging thousands of random games gave a shockingly usable estimate in Go.

It is a disaster in chess, for a reason worth stating precisely: **chess outcomes are determined by short forcing sequences, and random play destroys them**. From a position where you are a queen up, random play throws the queen back within a few moves; from a position with mate in one, random play finds the mate one time in thirty. The rollout's answer is dominated by noise that has nothing to do with the position.

Confidence: established

AlphaGo (2016) replaced the rollout with a learned **value network**, and AlphaZero (2017) dropped rollouts entirely. Leela follows AlphaZero: phase 3 is one forward pass returning (P, V), and no game is ever played out. The statistics are still Monte-Carlo — we are averaging samples — but the samples are network evaluations of concrete positions chosen deliberately, not random games.

Where people go wrong

Because of this, the phrase "Leela plays out thousands of games in its head" — common in chess commentary and present in the earlier draft guide this book replaces — is simply false. Leela never plays a game out. At 800 nodes it has looked at 800 positions, evaluated each once, and averaged. If your tool ever says "simulations", make sure the user cannot infer rollouts from it.

5.3 Backpropagation and the sign flip

Phase 4 is where the bugs live. A value is always *from the point of view of the player to move at that node* (Chapter 1, negamax). So as the value walks up the tree, it must alternate sign at every ply.

for each edge (s, a) on the path, from the leaf upwards: $N(s,a) \mathrel{+}= 1, \qquad W(s,a) \mathrel{+}= v, \qquad v \leftarrow -v.$ (5.1)

Worked example 5.1 — backing up a value through three plies

The search selected: root (s_0, White to move) → move a → s_1 (Black to move) → move b → s_2 (White to move) → move c → new leaf s_3 (Black to move). The network evaluates s_3 at $V(s_3) = -0.6$ — *Black to move there, and Black is worse*, since values are always for the side to move.
Now walk up, flipping each time:

Edge	Whose node	Value added to W	Meaning
(s_2, c)	White to move at s_2	$-(-0.6) = +0.6$	good for White
(s_1, b)	Black to move at s_1	$-(+0.6) = -0.6$	bad for Black
(s_0, a)	White to move at s_0	$-(-0.6) = +0.6$	good for White

Every N on the path also increments by 1. Sanity check the top and bottom: a position that is bad for Black-to-move must be good for White at the root, and indeed both White nodes recorded +0.6. If you ever compute a chain where a single leaf makes both sides happy, you have a sign bug.

Key idea

Q is always relative to the side to move at the parent node. When you read `Q: 0.94` on an edge out of a node where Black is to move, it means the move is excellent *for Black*. This trips up everyone reading engine output for the first time, and it is the single most common source of an evaluation displayed with the wrong sign in a chess GUI.

We can verify the convention on real output rather than take my word for it:

Real engine outputtitle

The king-and-pawn position (White Ke6, pawn e5, Black Ke8) evaluated with White to move and then with Black to move — the identical board, one tempo apart:

	`WL` at root	best move	Stockfish d30
White to move	+0.942	Kd6	mate in 12 for White
Black to move	−0.948	Kf8	mate in 16 for White

Both numbers say "White is winning". The first says it as +0.94 (White to move, White happy); the second says it as −0.95 (Black to move, Black unhappy). The sign flipped because the mover changed, not because anything on the board did.

5.4 UCT: UCB1 at every node

Put UCB1 (Equation (4.4)) on each node and you have UCT. The selection rule at node s is

$$a^* \;=\; \arg\max_a \left[Q(s,a) \;+\; c\sqrt{\frac{\ln N(s)}{N(s,a)}} \right], \tag{5.2}$$

applied repeatedly from the root until you fall off the bottom of the tree, which is phase 1. An unvisited edge has $N(s,a) = 0$ and therefore infinite bonus, so *every legal move at a node must be tried once before any is tried twice.*

That last sentence is the whole problem.

5.4.1 Why UCT alone cannot play chess

Worked example 5.2 — the cost of "try everything once"

Your corpus says $b \approx 31$ legal moves per position (Chapter 1). UCT must visit every child once before deepening anywhere. So:

To have looked once at every line of depth…	costs (nodes)	
1 ply	31	one visit each to your own moves
2 ply	961	
3 ply	29,791	
4 ply	923,521	

A search budget of 800 nodes — the kind this project uses for a live board — cannot complete *two plies*. It would spend its first 31 visits on the root's children, including every move that hangs a queen, and then have 769 left to distribute over a tree whose second layer alone needs 961.

And the visits it does spend are spent stupidly. UCB1 has no way of knowing that 1.g4 deserves less attention than 1.e4 *before measuring it.* It must pay a full visit to discover what any club player knows at a glance.

Key idea

UCT's fatal assumption is that **all options start equal**. That is right for slot machines — they are anonymous — and catastrophically wrong for chess moves, which come with enormous amounts of prior evidence about which ones are worth examining. The fix is not a better bonus. The fix is to **stop starting from ignorance**.

5.4.2 What the fix has to look like

Before turning the page, notice how strong the constraints are on any repair:

1. It must let the search *skip* moves entirely at low visit counts — with 800 nodes and 31 legal moves, most moves must receive zero visits, forever.
2. It must nevertheless be able to *come back* to a skipped move if the favoured ones fail. A hard "only look at the top 3" cutoff cannot do this, and would make the engine blind exactly where blindness is fatal.

3. It must degrade gracefully into UCT-like behaviour as visits grow, so that with enough compute the prior's opinion can be overruled by measurement.

Requirement 2 is the interesting one, and it is the reason the answer is not simply "pre-filter the move list". Chapter 6 builds a bonus that satisfies all three — and the move that comes back from the dead in our real data is a queen sacrifice that the network initially rated at 1.6%.

in this repository

The four phases are the loop inside `engine/lc0.exe`; you never see them directly, but every number in `-verbose-move-stats` is produced by them. When this project's backend requests a search and reads back visit counts and principal variations, it is reading the state of the tree that this loop left behind.

Exercises

Exercise 5.1 *(the loop)* Write out the four phases in your own words, in one sentence each, without looking. Then say which phase the neural network participates in, and how many times it is called per iteration.

Exercise 5.2 *(sign flips)* A selection path runs root (White) $\to s_1$ (Black) $\to s_2$ (White) $\to s_3$ (Black) $\to$ new leaf s_4 (White), and the network reports $V(s_4) = +0.35$. Write out the value added to W on each of the four edges, with signs, and say in words what the root edge's contribution means.

Exercise 5.3 *(reading a Q)* An edge out of a Black-to-move node shows `N: 210 (Q: 0.77)`. Is Black doing well or badly? A GUI displays this as "+0.77 White". What has gone wrong, and where in Equation (5.1) did it go wrong?

Exercise 5.4 *(rollouts)* Estimate, roughly, the probability that a random legal continuation from a position with mate in one actually plays the mate, assuming 31 legal moves. Then estimate the probability that a 40-ply random rollout preserves a decisive queen advantage. Use these to argue in two sentences why rollouts died.

Exercise 5.5 *(UCT's cost)* With $b = 31$, how many nodes does UCT need before it has visited every 3-ply line once? Express the answer as a multiple of an 800-node budget. Now redo it for the endgame branching factor $b = 15.8$ from your corpus, and comment.

Exercise 5.6 *(the first sweep)* Under pure UCT with 800 nodes at $b = 31$, what fraction of the budget is consumed just by giving every root move its first visit? If 20 of those 31 moves are obviously bad to a club player, how much of the budget was wasted, and what would you have to know in advance to avoid the waste?

Exercise 5.7 *(designing the repair)* Before reading Chapter 6: propose a modification of Equation (5.2) that uses a prior probability $P(a \mid s)$. Write down your formula. Then check it against the three requirements in §5.5.2 and note which ones it fails. (Keep this; you will compare it with the real answer.)

Exercise 5.8 *(the hard cutoff)* Suppose you implement "only ever search the top 3 moves by prior". Describe a concrete chess situation in which this loses a game outright, and explain which of the three requirements it violates. Why is this failure mode particularly dangerous for a *training* tool as opposed to a playing engine?

Exercise 5.9 *(budget and depth)* If a search concentrates 90% of its visits on one move at each ply, how deep can 800 nodes reach along the main line? Compare with the UCT answer from Exercise 5.5. This is a preview of the actual mechanism — state in one sentence what the search must be buying with that concentration.

Chapter 6

Adding a prophet: building PUCT one failed attempt at a time

After this chapter you can

- Say what a policy prior is and what it is not.
- Build the PUCT selection rule through five attempts, and be able to say which failure each surviving symbol repairs.
- Prove the equilibrium property: when search learns nothing, visits become proportional to the prior — and check it against real output.
- Compute $U(s, a)$ exactly as Leela computes it, including the growth of c_{puct} and the first-play-urgency value of an unvisited move.
- Verify your arithmetic against the engine's own `S` column to five decimals.

6.1 The prophet

Chapter 5 ended stuck: UCT must taste every move before it can prefer any, and at $b \approx 31$ that is unaffordable. What we want is what a strong player has — a glance that says *these two moves, probably; that one, conceivably; the other twenty-eight, no.*

Assume we have one. Formally, a **policy prior** is a probability distribution over the legal moves of a position:

$$P(a \mid s) \geq 0, \qquad \sum_{a \in \mathcal{A}(s)} P(a \mid s) = 1.$$

Where it comes from is Part IV's problem; here it is an oracle we may consult for free. What it means is worth being exact about, because the whole design depends on it:

Key idea

$P(a \mid s)$ is **not** an estimate of how good the move is. It is an estimate of *how likely this move is to be the one finally played* — literally, what fraction of the time a strong player (in Leela's case, its own trained self) would choose it. It is a **claim about attention**, not about value. A move can be excellent and have low P if it is the sort of move that is usually wrong.

That last sentence is not a technicality. It is the mechanism behind everything your project calls a "hidden gem", and here it is in real numbers before we start:

Real engine outputtitle

Position after 15. . . Nxd7 in Morphy–Brunswick/Isouard, Paris 1858. 46 legal moves. Top priors from the network in `engine/791556.pb.gz`, no search:

Move	$P(a \mid s)$	
Qb7	31.93%	the network's instinct
Qb5	9.57%	
Qc3	8.33%	
Qxe6+	7.92%	
Rxd7	7.81%	
Qa4	7.65%	
⋮	⋮	
Qb8+	**1.60%**	**Stockfish (depth 30): mate in 2**

The move that forces mate in two gets 1.60% of the network's attention. It is not ranked first, second or third. Any design that decides in advance which moves are worth looking at must survive this position.

6.2 Five attempts

We want a selection rule of the shape $\arg\max_a [\, Q(s,a) + U(s,a) \,]$ where U now uses P. Chapter 5 set three requirements: (R1) most moves may get *zero* visits; (R2) any move must still be reachable later; (R3) with enough visits, measurement must be able to overrule the prior.

Attempt 1: filter the move list — search only the top k priors

Simple, fast, and used by real programs in other domains. Take $k = 3$ in the Morphy position: we search Qb7, Qb5, Qc3, and **mate in two is not in the list**. Not at 800 nodes, not at eight billion.

Fails R2, catastrophically and silently. A hard cutoff converts the prior's mistakes into the engine's permanent blind spots — and the moves a policy under-rates are exactly the surprising ones, which are exactly the ones a training tool exists to show you.

Attempt 2: add the prior to the value — $Q(s,a) + c\,P(a \mid s)$

Now every move is reachable in principle. But the prior's contribution never decays, so it is not a *bonus for uncertainty*, it is a permanent thumb on the scale.

With $c = 1$ in the Morphy position: Qb7 gets a standing +0.319 and Qb8+ a standing +0.016. Suppose the search does 100,000 visits and establishes the true values — Qb8+ wins by force, $Q = 1.0$; Qb7 is merely much better, $Q \approx 0.66$. Scores: Qb8+ gives 1.016, Qb7 gives 0.979. Here it happens to work, because the truth gap (0.34) exceeds the prior gap (0.30). Make the position one where the best move wins by a small margin and the prior wins permanently.

Fails R3: no amount of evidence can wash the prior out. And it fails R1 too, in the other direction — because the term is a constant, the search has no reason to *stop* returning to a well-measured move.

Attempt 3: multiply the prior into UCB1's bonus — $Q + c\,P\sqrt{\dfrac{\ln N}{n}}$

Now we are close to something sensible: the bonus decays with visits (good), grows slowly with total search (good), and is scaled by how promising the move looked (new and good). One fatal defect remains, and it is the same one that sank UCT: at $n = 0$ the term $1/\sqrt{n}$ is **infinite**. Every legal move still gets a mandatory first visit, whatever its prior. We are back to spending 31 visits before the second look at anything.
Fails R1. The fix must make the bonus *finite* at zero visits.

Attempt 4: make it finite at zero — $Q + c\,P\dfrac{\sqrt{\ln N}}{1+n}$

Replacing $1/\sqrt{n}$ with $1/(1+n)$ does exactly what is needed: at $n = 0$ the bonus is $c\,P\sqrt{\ln N}$, which is *small for a move with a small prior*. A move with $P = 0.4\%$ starts with a small claim on the budget, and the search is free to ignore it. R1 satisfied.
But now look at the cost of that change. The denominator went from $\sqrt{n}$ to n, so the bonus decays much faster than before. Meanwhile the numerator still grows only like $\sqrt{\ln N}$ — glacially: from $N = 100$ to $N = 10{,}000$ it grows by a factor of $\sqrt{\ln 10^4 / \ln 10^2} = \sqrt{2} = 1.41$. Fast decay plus glacial growth means a neglected move's priority stays flat essentially forever. In the Morphy position, Qb8+'s bonus would creep up by 41% over a hundredfold increase in search — while it needs to grow by a factor of about ten to become competitive.
Fails R2 in practice, if not quite in theory. **The numerator has to grow faster.**

Attempt 5: let the numerator grow like $\sqrt{N}$

$$U(s,a) = c_{\text{puct}}\; P(a \mid s)\; \frac{\sqrt{N(s)}}{1 + N(s,a)}.$$

Now, from $N = 100$ to $N = 10{,}000$ the numerator grows tenfold, not by 41%. A move passed over early is not passed over forever: its claim strengthens steadily as the search pours visits elsewhere. R2 satisfied — and we will watch it happen, in real output, in §6.5.

PUCT — the rule Leela actually uses

$$a^* \;=\; \arg\max_a \left[\underbrace{Q(s,a)}_{\text{what I measured}} \;+\; \underbrace{c_{\text{puct}}\; P(a \mid s)\; \frac{\sqrt{N(s)}}{1 + N(s,a)}}_{U(s,a)\text{: what I haven't checked}} \right] \tag{6.1}$$

with $N(s) = \sum_b N(s,b)$ the total visits already spent at this node.

Read the three moving parts against the three failures they repair:

Symbol	Says	Repairs
$P(a \mid s)$	"this much of my attention is owed to this move"	Attempt 1's blindness
$\dfrac{1}{1+N(s,a)}$	"…divided by how much I have already looked"	Attempt 3's mandatory first visit
$\sqrt{N(s)}$	"…multiplied by how much thinking has happened here"	Attempt 4's frozen-out move

Key idea

U is a **claim on the budget**. Read $P\sqrt{N}/(1+n)$ as: *"I was promised roughly a P share of the attention here; N units of attention have been spent; I have had n. How overdue am I?"* The whole formula is one sentence: **take the move with the best combination of proven performance and unpaid attention.**

6.3 The equilibrium property: visits become the policy

Here is a small theorem that makes PUCT much easier to think about — and, unusually for this subject, it is two lines.

Confidence: established

Claim. Suppose the search learns nothing: every move's Q is identical (call it q). Then PUCT allocates visits in proportion to the prior, $N(s,a) \propto P(a \mid s)$.

Proof. Selection always picks the maximum of $Q+U$. If all Q are equal to q, the maximum of $q+U$ is the maximum of U. A rule that repeatedly picks the largest U and thereby decreases it drives all the U's towards equality. So at equilibrium, for any two moves a and b,

$$c_{\text{puct}} P_a \frac{\sqrt{N}}{1+n_a} = c_{\text{puct}} P_b \frac{\sqrt{N}}{1+n_b} \implies \frac{P_a}{1+n_a} = \frac{P_b}{1+n_b} \implies 1+n_a \propto P_a. \quad \blacksquare$$

Key idea

The visit distribution is the prior, reshaped by what search discovered. If search finds nothing, visits reproduce the policy exactly. Every departure of the visit distribution from the policy distribution is *information the search added.* That single sentence is the foundation of Chapter 9, of your project's "optical trap" and "hidden gem" categories, and of how Leela trains itself (Chapter 12).

We can test the claim on real output. Take a position where the search genuinely learns nothing — a dead-drawn king-and-pawn ending in which every move evaluates to exactly 0.000:

Real engine outputtitle

White Ke4, pawn e5, Black Ke6 (Black holds the opposition). Stockfish at depth 30 confirms 0.00 for every move. Leela's search after 800 nodes:

Move	P	N	share of visits	Q
Kf4	19.82%	172	21.5%	0.0059
Kd4	19.74%	171	21.4%	0.0059
Kd3	20.45%	155	19.4%	0.0000
Kf3	20.19%	152	19.0%	0.0000
Ke3	19.79%	148	18.5%	0.0000

Priors are nearly uniform ($\approx 20\%$ each) and so are the visits ($\approx 20\%$ each), as the theorem predicts. The residual tilt towards Kf4 and Kd4 is not noise: those two moves have $Q = 0.0059$ against 0.0000 for the rest — a difference in the fifth decimal place of a drawn position, and the search still spent 3% more of its budget on them. The mechanism is exactly as advertised, down to a rounding error's worth of "advantage".

And now the same test where the search *does* learn something:

Real engine outputtitle

Move	P	N	visit share	Q	share − prior
e4	22.22%	532	35.6%	0.1058	+13.4 pts
d4	18.41%	451	30.2%	0.1062	+11.8 pts
Nf3	14.22%	230	15.4%	0.0995	+1.2 pts
c4	9.19%	100	6.7%	0.0849	−2.5 pts
g3	7.26%	64	4.3%	0.0733	−3.0 pts
e3	4.43%	28	1.9%	0.0483	−2.5 pts

Here the Q's differ, so the visit distribution is a **sharpened** version of the prior: the top two moves take a larger share than their priors promised, and everything below Nf3 takes less. Search has concentrated attention on what measurement supports. That sharpening is the search's entire contribution, and Chapter 12 shows it is also precisely the training signal that improves the network.

6.4 The two remaining pieces of real arithmetic

Equation (6.1) is the formula in every paper. The engine on your disk uses two refinements, and you need both to reproduce its numbers exactly — which we are about to do.

6.4.1 c_{puct} grows with the search

Leela does not use a constant. It uses

$$c_{\text{puct}}(N) \;=\; \texttt{CPuct} \;+\; \texttt{CPuctFactor} \cdot \ln\!\left(\frac{N + \texttt{CPuctBase} + 1}{\texttt{CPuctBase}}\right), \tag{6.2}$$

with the defaults on your engine being `CPuct` = 1.745, `CPuctBase` = 38739, `CPuctFactor` = 3.894 (read off `lc0.exe`'s own `uci` option list).

The logarithm means that for small searches the growth is negligible — at $N = 182$, $c_{\text{puct}} = 1.745 + 3.894 \ln(1.00470) = 1.745 + 0.018 = 1.763$ — while at $N = 10^6$ it reaches about 14. The design intent is legible: a small search should trust the network, and a huge search should be increasingly willing to look at what the network dismissed. (Compare your answer to Exercise 4.8.)

6.4.2 First-play urgency: what Q does an unvisited move have?

Equation (6.1) needs $Q(s,a)$ for every candidate — including moves never visited, which have no average at all. What value should they be given?

- *Optimistic* ($Q = +1$): every move looks winning until checked, so everything gets visited once. That is UCT again.
- *Neutral* ($Q = 0$): reasonable, but wrong in won and lost positions — in a position where you are winning by a queen, "0" is a pessimistic slander of every unexamined move; in a lost position it is wild optimism.
- *Inherit the parent* ($Q = Q(s)$): assume a move is as good as the position it comes from — sensible, but then unvisited moves tie with measured ones and get pulled in too eagerly.

Leela's default, **FPU reduction**, takes the third option and subtracts a penalty that grows with how much of the prior has already been explored:

$$Q_{\text{FPU}}(s) \;=\; Q(s) \;-\; \texttt{FpuValue} \cdot \sqrt{\textstyle\sum_{b:\,N(s,b)>0} P(b \mid s)}, \qquad \texttt{FpuValue} = 0.33. \tag{6.3}$$

The reasoning: if you have already examined moves covering 80% of the prior mass and none of them beat what you have, the remaining 20% is probably not hiding a better move — so discount it. Early in a node's life the penalty is near zero; late in its life it is substantial.

Worked example 6.1 — deriving FPU from real output — a piece of detective work

Leela's verbose output prints `Q: 0.0` for an unvisited move (there is no average to print), but its `S` column is the real PUCT score, and $\texttt{S} = Q_{\text{FPU}} + U$. So we can recover the hidden value the engine actually used.
In the Morphy position at $N(s) = 182$, the line for Qb8+ reads $\texttt{U} = 0.37909$, $\texttt{S} = 0.69780$. Therefore

$$Q_{\text{FPU}} = \texttt{S} - \texttt{U} = 0.69780 - 0.37909 = 0.31871.$$

Check it against Equation (6.3). The root's own value is $Q(s) = 0.6117$, and the engine helpfully prints the visited prior mass on its root line: `P: 79.96%`. So

$$Q(s) - 0.33\sqrt{0.7996} \;=\; 0.6117 - 0.33(0.8942) \;=\; 0.6117 - 0.2951 \;=\; 0.3166.$$

Against the recovered 0.31871 — agreement to three decimals, the residue being the rounding of the printed percentages. **Equation (6.3) is confirmed on real output.**

6.5 Watching R2 work: the queen sacrifice comes back

We can now close the loop on the requirement that killed three of our five attempts. Recall Qb8+ has a prior of 1.60% and forces mate. Here is what PUCT does with it, from the actual engine:

Real engine outputtitle

At $N(s) = 182$ visits (search still running):

Move	P	N	Q	U	$S = Q + U$
Qb7	31.93%	169	0.6575	0.0445	**0.70199**
Qb8+	**1.60%**	**0**	0.3187*	0.3791	**0.69780**
Rxd7	7.81%	4	0.3155	0.3701	0.68387
Qc3	8.33%	2	−0.0005	0.6583	0.65774
Qb5	9.57%	2	−0.1019	0.7565	0.65235

* the FPU value recovered in §6.4.2; the engine's `Q` column prints 0.0 for unvisited moves.

The mating move, which the network rated at 1.60% and which has **never been looked at**, is ranked *second* — behind the leader by 0.004. Its entire claim comes from the U term: 169 visits have piled into Qb7, and $\sqrt{N}/(1+n)$ has been quietly compounding.
Six visits later, the search selects it. The result:

At $N(s) = 188$	P	N	Q	U	S
Qb8+	1.60%	**3**	**1.0000**	0.0964	**1.09636**
Qb7	31.93%	172	0.6587	0.0445	0.70310

$Q = 1.0$ exactly: not an estimate but a proof — 16.Qb8+ Nxb8 17.Rd8# is a terminal sequence, so the value is certain rather than sampled. The engine reports `bestmove b3b8` and *stops searching*, at 188 nodes out of a requested 6400.

Key idea

Three visits beat one hundred and seventy-two. This is worth dwelling on, because it is where the naive reading of Monte-Carlo search — "the most-visited move wins" — breaks. A proven result outranks a well-sampled estimate. Chapter 9 deals with the general question of which number decides the move, and Chapter 8 with how certainty propagates.

And notice what would have happened under each rejected attempt: Attempt 1 (top-3 filter) never sees the move. Attempt 2 (constant prior bonus) gives it a standing +0.016 against Qb7's +0.319 — it would need the search to have already discovered the truth it is supposed to be discovering. Attempt 4 ($\sqrt{\ln N}$ numerator) reaches $U \approx 0.04$ instead of 0.379 at the same point, ranking Qb8+ somewhere in the twenties. **The $\sqrt{N}$ is the reason the mate was found.**

Where people go wrong

Do not over-generalise this happy ending. PUCT guarantees that a low-prior move's claim *grows*, not that it grows *in time*. If the prior had been 0.05% rather than 1.60%, the same computation gives $U \approx 0.012$ at $N = 182$, and the move would need roughly a thousand times more search to surface. Leela does miss things, and the shape of what it misses is precisely "moves its policy rates near zero in positions where nothing else is failing". Chapter 15 is about whether we can detect those cases from inside the network instead of waiting for search to stumble on them.

in this repository

P is the `policy` array this project already extracts and draws as arrows on the board; `U` and `S` are visible in `-verbose-move-stats`. The "policy-blindness" metric in your diagnosis layer is a statement about P; the "optical trap / hidden gem" classification is a statement about the gap between P and the visit distribution, i.e. about how much of §6.3's sharpening happened and in which direction.

Exercises

Exercise 6.1 *(what P means)* In one sentence each, state what $P(a \mid s) = 0.32$ claims and what it does *not* claim. Then explain how a move can have $P = 1.6\%$ and be the only move that wins.

Exercise 6.2 *(compute U)* A node has $N(s) = 100$ total visits. Move a has $P = 0.25$ and $N(s, a) = 40$; move b has $P = 0.05$ and $N(s, b) = 0$. Using $c_{\text{puct}} = 1.745$, compute U for both. Which is larger, and by what factor?

Exercise 6.3 *(the crossover)* Continue the previous exercise: move a has $Q = 0.55$ and move b has $Q_{\text{FPU}} = 0.30$. Compute S for both. Now recompute at $N(s) = 400$ with a having absorbed all the extra visits ($N(s, a) = 340$). At which of the two search sizes does b get selected? State the mechanism in one sentence.

Exercise 6.4 *(verify the engine)* From the real table in §6.5, take the Qb7 line at $N(s) = 182$: $P = 0.3193$, $N(s, a) = 169$. Compute U using Equation (6.1) with c_{puct} from Equation (6.2). Compare with the engine's 0.04453. How many decimal places do you match?

Exercise 6.5 *(the equilibrium theorem)* Reproduce the proof of §6.3 in your own words. Then: a node has three moves with priors $0.6, 0.3, 0.1$ and all Q equal. After 100 visits, what does the theorem predict for n_1, n_2, n_3? (Remember it is $1 + n_a$ that is proportional to P_a; discuss whether the "+1" matters at 100 visits and at 3 visits.)

Exercise 6.6 *(sharpening)* Using the real 1600-node table for the initial position, compute for each move the ratio (visit share)/(prior). Which moves have ratio above 1? Explain what a ratio of 1.60 for e4 and 0.43 for e3 tells you, in terms of §6.3.

Exercise 6.7 *(FPU by hand)* A node has $Q(s) = -0.20$ and has visited moves covering 45% of the prior mass. Compute Q_{FPU} for its unvisited moves. Now suppose the visited mass rises to 90%: recompute. Explain in one sentence why the penalty should grow.

Exercise 6.8 *(FPU's effect on style)* Suppose you set `FpuValue` to 0.0 (unvisited moves inherit the parent's value untouched). Describe how the search's behaviour changes: broader or narrower? Now set it to 1.0. Which setting would make an engine more likely to find a surprising sacrifice, and what does it cost?

Exercise 6.9 *(c_{puct} growth)* Using Equation (6.2) with your engine's defaults, compute c_{puct} at $N = 800$, $N = 10^5$, and $N = 10^7$. Plot roughly. What does the designers' choice say about how much to trust the policy at each scale, and does it agree with the answer you gave in Exercise 4.8?

Exercise 6.10 *(the hidden move's timetable)* A move has $P = 0.005$ and $N(s, a) = 0$ in a node where the leader holds $Q = 0.60$ and the FPU value is 0.30. Using $c_{\text{puct}} \approx 1.75$, solve for the value of $N(s)$ at which the unvisited move's S overtakes the leader's Q. Comment on whether that node count is reachable in your tool's configuration, and what this implies for what the tool can promise a user.

Exercise 6.11 *(design)* Compare Equation (6.1) with the formula you invented in Exercise 5.7. Name one thing yours does better and one thing it does worse. (This is not a trick question — several published variants differ from PUCT in defensible ways.)

Chapter 7

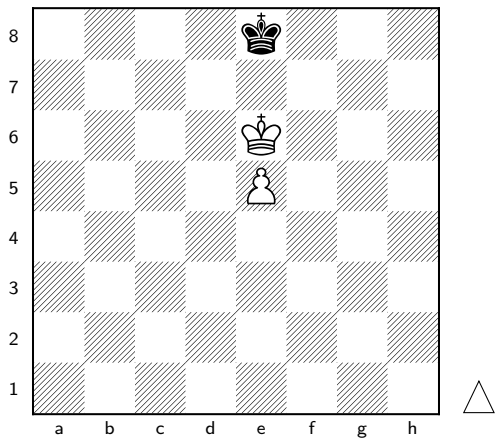
A complete search by hand

After this chapter you can

- Execute eight full iterations of Leela's search with pencil and paper — selection, expansion, evaluation, backpropagation — and match the engine's own numbers to five decimal places.
- Watch the search deepen along its main line without being told to.
- Watch a refutation happen: a move examined once, scored 0.000, and abandoned — with independent confirmation that abandoning it was correct.
- Explain which number finally chooses the move played, and why it is not Q.
- Discover, by experiment, that evaluating a position from a bare FEN is not the same as evaluating it in a game.

7.1 The position, and the truth about it

Everything in this chapter is real. The priors, the values, the selections and the resulting tree are what `engine/lc0.exe` with `791556.pb.gz` actually did on this machine; the arithmetic below is reproduced by `tools/simulate_search.py`, which checks itself against the engine and refuses to disagree with it.



White: Ke6, pawn e5. Black: Ke8. **White to move.** Four legal moves: Kd6, Kf6, Kd5, Kf5 (the pawn cannot advance — its own king is in front of it, and d7, e7, f7 are all controlled by

the black king).

We chose this position for one reason: with four legal moves the whole tree fits on a page. But it is not a toy — it contains a real trap, and here is the independent ground truth, from Stockfish 16.1 at depth 30 (*not* from Leela, and not from the author):

Move	Stockfish depth 30	
Kd6	**mate in 12**	wins
Kf6	**mate in 18**	also wins, less efficiently
Kd5	0.00	**throws the win away**
Kf5	0.00	**throws the win away**

So two of the four moves win and two draw. Keep that in your pocket; the search does not know it yet.

7.2 The bookkeeping

Every iteration we recompute, for each of the four moves,

$$S(a) \;=\; \underbrace{Q(a)}_{\text{measured, or FPU if unmeasured}} \;+\; \underbrace{c_{\text{puct}}\,P(a)\,\frac{\sqrt{N}}{1+n_a}}_{U(a)}$$

and select the largest. The constants, from Chapter 6:

$$c_{\text{puct}}(N) = 1.745 + 3.894\ln\frac{N+38740}{38739}, \qquad Q_{\text{FPU}} = Q(\text{root}) - 0.33\sqrt{\textstyle\sum_{\text{visited}} P},$$

and N means the total visits already spent on the root's children (using $\sqrt{\max(N,1)}$ so that the first selection is decided by the priors alone).

The network's opening statement about this position — one forward pass, no search:

Real engine outputtitle

$$V(\text{root}) = +0.97602, \qquad d = 0.024$$

Move	$P(a \mid s)$	
Kd6	45.13%	
Kf6	44.23%	
Kf5	5.38%	one of the two moves that draws
Kd5	5.26%	the other

Note what the policy has and has not done. It has correctly identified that the two king-forward moves are the serious ones — 89% of its attention goes there. But it still assigns 10.6% to two moves that throw away a won game. **The policy is a good guess, not an oracle**; the search's job in this position is to spend a little of its budget converting that 10.6% of doubt into knowledge.

7.3 The eight iterations

Do these with a calculator. Each table is the state *before* the selection.

Iteration 1 — nothing measured

$N = 0$ (so $\sqrt{\max(N, 1)} = 1$), no move has been visited, so all four get $Q_{\text{FPU}} = Q(\text{root}) = 0.97602$ (nothing visited, so the penalty $0.33\sqrt{0} = 0$).

$$U(\text{Kd6}) = 1.7451 \times 0.4513 \times \frac{1}{1+0} = 0.78756.$$

Move	n_a	Q	U	S
Kd6	0	0.97602	0.78756	**1.76358**
Kf6	0	0.97602	0.77186	1.74788
Kf5	0	0.97602	0.09389	1.06991
Kd5	0	0.97602	0.09179	1.06781

With every Q identical, the ranking is purely the priors — exactly as Chapter 6 predicted. **Select Kd6.** Expand it, ask the network: the child (Black to move) evaluates to -0.96766 from Black's point of view, so the root edge Kd6 receives $+0.96766$.

Iteration 2 — the first measurement, and the FPU penalty appears

Now one move is measured and three are not. The root's own Q has been updated to $(0.97602 + 0.96766)/2 = 0.97184$, and the visited prior mass is 0.4513, so

$$Q_{\text{FPU}} = 0.97184 - 0.33\sqrt{0.4513} = 0.97184 - 0.22169 = 0.75015.$$

Move	n_a	Q	U	S	
Kf6	0	0.75015	0.77190	**1.52205**	FPU
Kd6	1	0.96766	0.39380	1.36146	measured
Kf5	0	0.75015	0.09389	0.84404	FPU
Kd5	0	0.75015	0.09180	0.84195	FPU

Key idea

Kd6 has the *best* measured value (0.96766 against a mere FPU guess of 0.75015) and is *not* selected. Its U halved the moment it was visited — the denominator went from $1 + 0$ to $1 + 1$. This is the anti-greedy mechanism of Chapter 4 doing its work: measuring something makes it less urgent to measure again.

Select Kf6. Expand: value $+0.98598$ to the root edge.

Iteration 3 — the first genuine contest

Two measured moves, both excellent. $Q(\text{root}) = 0.97655$; visited prior mass $0.4513 + 0.4423 = 0.8936$, so $Q_{\text{FPU}} = 0.97655 - 0.33(0.94530) = 0.66460$.

Move	n_a	Q	U	S
Kf6	1	0.98598	0.54585	**1.53183**
Kd6	1	0.96766	0.55696	1.52462
Kf5	0	0.66460	0.13279	0.79739
Kd5	0	0.66460	0.12983	0.79443

The two leaders are separated by 0.007; the two draws have fallen a full 0.7 behind, because their FPU value dropped when the good moves got measured and came back healthy. **Select Kf6**, and here something new happens: Kf6 already has a child node, so selection does not stop at the root. It recurses. At the Kf6 node (Black to move, three legal replies with priors Kf8 52.05%, Kd7 25.89%, Kd8 22.05%), the same rule picks **Kf8**, which is expanded and evaluated. The value returned to the root edge Kf6 is +0.95129.

Worked example 7.1 — following the sign flips on iteration 3

The new leaf is the position after 1.Kf6 Kf8, *White* to move. Its network value is +0.95129 from White's point of view. Walking up:

- edge (Kf6-node, Kf8), where *Black* is to move: receives −0.95129 — Black does not like it;
- edge (root, Kf6), where *White* is to move: receives +0.95129.

Kf6's running average becomes $(0.98598 + 0.95129)/2 = 0.968635$.

Iterations 4–8 — the pattern

The remaining iterations are the same arithmetic; here is the whole run in one table. "Line" is the path the selection walked before expanding a new node, and "backs up" is the value that returned to the root edge.

It.	Selected	S of the two leaders		Line walked	backs up	root edge after
		Kd6	Kf6			
1	Kd6	1.76358	1.74788	Kd6	+0.96766	Kd6: n=1, Q=0.96766
2	Kf6	1.36146	1.52205	Kf6	+0.98598	Kf6: n=1, Q=0.98598
3	Kf6	1.52462	1.53183	Kf6 Kf8	+0.95129	Kf6: n=2, Q=0.96863
4	Kd6	1.64983	1.41434	Kd6 Kd8	+0.99759	Kd6: n=2, Q=0.98262
5	Kd6	1.50779	1.48333	Kd6 Kf7	+0.99992	Kd6: n=3, Q=0.98839
6	Kf6	1.42878	1.54411	Kf6 Kf8 e6	+0.97860	Kf6: n=3, Q=0.97196
7	Kd6	1.47084	1.44478	Kd6 Kd8 e6	+0.97060	Kd6: n=4, Q=0.98394
8	Kf6	1.40085	1.48270	…		

Real engine outputtitle

After seven backed-up values, our pencil-and-paper tree says:

	by hand		`lc0.exe, go nodes 8`	
Move	n_a	Q	n_a	Q
Kd6	4	0.98394	4	0.98394
Kf6	3	0.97196	3	0.97196
Kf5	0	0.66651	0	0.66652
Kd5	0	0.66651	0	0.66652

Every visit count identical; every average identical to five decimal places (the last digit of the FPU values differs because the priors above are printed to four figures, not because anything conceptual is missing). **You have just run Leela's search by hand.**

7.4 Two things you should notice

7.4.1 The search deepened without being told to

Nobody wrote "now go deeper". Look at the "line walked" column: iterations 1–2 stop at the root's children, 3–5 reach two plies, 6–7 reach three. Depth is an *emergent* property of U collapsing where visits pile up. A node visited n times has its U divided by $1 + n$, so attention flows down the tree the way water flows downhill — always into the part of the tree that has been examined least relative to its promise.

Contrast this with alpha–beta, which searches to a fixed depth and then extends selectively by hand-written rules. Leela has no depth parameter at all. The principal variation is simply the path you get by always taking the most-visited child.

7.4.2 The two bad moves have not been touched

After eight iterations, Kd5 and Kf5 have $n = 0$. Their FPU value has drifted down from 0.976 to 0.667 while the good moves proved themselves; their U has crept up from 0.092 to 0.249 as the root accumulated visits. They are being ignored, but not forgotten — exactly the balance Chapter 6 was built to achieve.

So when does the search get to them? Run the engine at higher budgets and watch:

Real engine outputtitle

	n at 64 nodes		n at 128 nodes		n at 800 nodes	
Move	n	Q	n	Q	n	Q
Kd6	31	0.993	62	0.979	377	0.956
Kf6	24	0.953	58	0.963	240	0.927
Kf5	0	—	**1**	**0.000**	**1**	**0.000**
Kd5	0	—	**1**	**0.000**	**1**	**0.000**

Somewhere between 64 and 128 nodes, each of the two drawing moves received **exactly one visit**, returned $Q = 0.000$ — and was never visited again, not in the next 672 nodes.

That single visit is the whole story of this chapter in miniature:

Key idea

The network's *policy* did not know these moves were bad: it gave them 10.6% of its attention. The network's *value head* knew immediately: one look returned 0.000 with a draw probability of 1.000. And the *search* is the machinery that converts the second into a correction of the first, at a cost of two visits out of 800. Policy proposes; value disposes; search is the negotiation.

And Stockfish, asked independently, agrees that 0.000 is right: Kd5 and Kf5 draw. Two visits out of eight hundred were enough to permanently eliminate half the legal moves in the position, correctly.

Where people go wrong

Notice how easily this could have gone wrong, and what it depends on. It worked because the *value* head recognised the drawn structure in a single forward pass. In a position where the refutation is five moves deep rather than immediate, that first visit would have returned an encouraging number, and the search would have needed to build a subtree to find the truth. The engine's blind spots are precisely the positions where *both* heads are wrong at

once — and that is not a rare corner case, it is the definition of a difficult position. Chapter 9 shows how to spot it happening in your own output.

7.5 Which number chooses the move?

At 800 nodes the search has Kd6 with $n = 377$, $Q = 0.956$ and Kf6 with $n = 240$, $Q = 0.927$. The engine plays `bestmove e6d6` — Kd6. Here both criteria agree, so the position does not settle the question. Which one is it, in general?

The standard answer is **the most-visited move, not the highest** Q, and the reason is worth understanding because it is counter-intuitive.

Key idea

Q is an average over a subtree, and a subtree can be small. A move visited three times with $Q = 0.99$ has an average over three samples; a move visited 500 times with $Q = 0.95$ has survived five hundred attempts to refute it. High Q on low n usually means *the search has not yet found the objection* — indeed, if the search believed that Q, it would have kept visiting the move, and n would not be small. **Visit count is the search's considered opinion;** Q **is a measurement that may be young.**

The exception is certainty. In Chapter 6's Morphy position the engine played a move with $n = 3$ over a move with $n = 172$, because the Q of 1.0 was not an average at all — it was a proof (16.Qb8+ Nxb8 17.Rd8#). Engines track proven wins and losses separately from sampled values, and a proof outranks any amount of sampling. That is the only regular circumstance in which a low-visit move is played.

in this repository

This is the rule your tooling must follow when reporting "what the engine thought": rank candidate moves by visits, report Q alongside as the evaluation, and treat any move whose Q is excellent but whose n is small as *a question*, not an answer. It is also the right basis for the "how seriously did LC0 weigh this plan" language in your design documents — that phrase means visits.

7.6 An experiment: the position is not the whole input

One last thing, discovered while checking this chapter's numbers, which matters for any tool that feeds FENs to a network.

Evaluating the position after 1.Kf6 Kf8 gives different answers depending on how you ask:

Real engine outputtitle

How the position was given to the engine	V	d
`position fen 5k2/8/5K2/4P3/8/8/8/8 w - - 2 2`	+0.98838	0.012
`position fen 4k3/... w - - 0 1 moves e6f6 e8f8`	+0.95129	0.049

The same 32 squares, the same side to move, the same castling and en-passant rights — and the evaluation differs by 0.037, with four times the draw probability. The second number is the one the search used (it matches the backed-up value in iteration 3 exactly).

The cause is that Leela's input is not a position, it is a **position with history**: the network sees the last several board states as additional input planes. Given a bare FEN, the engine has

to fabricate a history by repeating the current position, and a repeated position is evidence of shuffling — hence the higher draw probability. We measured the same effect at one ply back in the previous chapter's data (0.96823 versus 0.96766).

Where people go wrong

This has a direct consequence for your project. Anywhere the backend evaluates a position by handing the engine a bare FEN — diagnosis of a game position, a drill, a cached critical point — it is asking a slightly different question from the one the engine answers during a game. The effect is small in quiet positions and largest where repetition matters: fortresses, perpetuals, drawn endgames. The fix is cheap: pass the move history (`position startpos moves ...`) rather than a FEN wherever the history is available. Worth an audit.

Exercises

Exercise 7.1 *(warm-up)* Reproduce iteration 1's U values for all four moves. Use $c_{\text{puct}} = 1.7451$ and check all four against the table.

Exercise 7.2 *(the FPU drop)* Compute Q_{FPU} at iteration 4, given $Q(\text{root}) = 0.97024$ and visited prior mass 0.8936. Why did the visited mass not change between iterations 3 and 4, even though a visit occurred?

Exercise 7.3 *(iteration 9)* Continue the simulation. Iteration 8 selects Kf6 (see the final row). Suppose it walks the line Kf6 Kf8 e6 Kg8 and backs up +0.982. Recompute $Q(\text{root})$, Q_{FPU}, all four U's, all four S's, and say which move iteration 9 selects. (Take $c_{\text{puct}} = 1.7459$.)

Exercise 7.4 *(when do the bad moves get looked at?)* At iteration 8 the drawing moves have $S = 0.915$ against the leader's 1.483. Using $U = c\,P\sqrt{N}/(1+0)$ with $P = 0.0538$, and assuming the leaders' S settles near 1.0 as U decays, estimate the value of N at which Kf5's S overtakes. Compare with the engine's actual answer (somewhere between 64 and 128 nodes) and comment on your estimate.

Exercise 7.5 *(the single visit)* Kf5 was visited once, returned 0.000, and was never visited again. Using the PUCT formula, show why: compute $U(\text{Kf5})$ at $N = 800$ with $n = 1$, add $Q = 0$, and compare with the leader's S. How large would N have to become before Kf5 is visited a second time?

Exercise 7.6 *(sign flips)* Iteration 6 walked Kf6 Kf8 e6 and the new leaf (Black to move) evaluated at -0.97860. Write out the value added at each of the three edges on the path, with signs and with whose point of view each represents.

Exercise 7.7 *(policy versus value)* The policy gave the two losing moves 10.6%; the value head scored them 0.000 on sight. In three sentences, explain why a network can be wrong in one head and right in the other about the same move. (You will be able to give a much better answer after Chapter 12; write this one now and compare.)

Exercise 7.8 *(most visits or best Q?)* A search reports move A with $n = 410$, $Q = 0.61$ and move B with $n = 12$, $Q = 0.88$. Which does the engine play, and why? Now suppose B's Q is exactly 1.000 and its subtree ends in mate. What changes, and why is that not an inconsistency?

Exercise 7.9 *(the history effect)* Design an experiment using your own corpus to measure how large the FEN-versus-history discrepancy is in practice: which positions would you sample, what would you compare, and what summary statistic would tell you whether it matters for the diagnosis pipeline? Predict the shape of the answer before you run it.

Exercise 7.10 *(a search that would fail)* Invent a (schematic, non-chess) situation in which this search finds the wrong move even with 10,000 nodes: specify priors, and the values returned by the first visit to each move, such that the true best move is never adequately examined. What single quantity in the PUCT formula would have to change to rescue it?

Exercise 7.11 *(report writing)* Write the three-sentence explanation your training tool should show a user about this position, using only quantities computed in this chapter. It must not contain a chess claim that did not come from the engine.

Part III

The Search as It Really Runs

Chapter 8

The details that make it work

(to be written)

Chapter 9

Reading a search tree like a chess player

(to be written)

Part IV

The Oracle: What the Network Is

Chapter 10

From a hand-made formula to a learned one

(to be written)

Chapter 11

Why attention: building a transformer layer

(to be written)

Chapter 12

The two heads, and the loop that trains them

(to be written)

Part V

Opening the Skull

Chapter 13

Probes: reading a mind, and the traps in claiming you did

(to be written)

Chapter 14

Attention maps: four ways to fool yourself

(to be written)

Chapter 15

Learned look-ahead and the priors that override it

(to be written)

Part VI

Bending the Engine

Chapter 16

Style as an objective function

(to be written)

Chapter 17

Sharpness that is real

(to be written)

Part VII

Making It Speak

Chapter 18

From numbers to sentences

(to be written)

Chapter 19

Capstone: one position, end to end

(to be written)

Appendices

Appendix A

Notation and symbols

(to be written)

Appendix B

Solutions to the exercises

(to be written)

Appendix C

Glossary

(to be written)

Appendix D

The engine data behind this book

(to be written)

Appendix E

Where each idea lives in this repository

(to be written)

Appendix F

Sources, and how much to trust each

(to be written)